Laporan Tugas Besar I Strategi Algoritma : Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Battlecode 2025

Idk-gnh

Fahd Muhammad Zahid	13524078
Al Farabi	13524086

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

BAB I : Deskripsi Tugas

1.1 Deskripsi Permainan Battlecode 2025

Battlecode 2025 merupakan sebuah kompetisi kecerdasan buatan berbasis permainan strategi waktu nyata yang dikembangkan oleh MIT (*Massachusetts Institute of Technology*). Dalam lingkungan ini, setiap tim ditantang untuk merancang algoritma kendali otomatis bagi sistem multi-agen yang beroperasi di dalam sebuah *grid* dua dimensi simetris dengan ukuran yang bervariasi antara 20x20 hingga 60x60 petak. Permainan ini menitikberatkan pada pengelolaan dua sumber daya utama, yakni *chips* yang berfungsi sebagai alat tukar untuk memproduksi unit, serta *paint* atau cat yang menjadi bahan bakar utama bagi robot untuk bergerak, menyerang, dan mengklaim wilayah.

Dalam ekosistem permainan ini, terdapat tiga jenis unit robot dengan spesialisasi peran yang berbeda. *Soldier* bertindak sebagai unit dengan ketahanan tinggi yang mampu membangun infrastruktur menara dan melakukan serangan jarak jauh. *Splasher* memiliki kemampuan unik untuk melakukan serangan area yang mampu mewarnai peta secara masif dalam waktu singkat, meskipun membutuhkan konsumsi cat yang sangat besar. Sebagai penyeimbang taktis, terdapat unit *Mopper* yang bertugas menghapus jejak cat musuh sekaligus melakukan sabotase dengan mencuri cadangan cat dari unit lawan.

1.2 Tujuan Tugas Besar

Pengerjaan Tugas Besar ini dirancang untuk mencapai beberapa tujuan kompetensi berikut :

1. Penerapan Algoritma Greedy :Mengimplementasikan konsep algoritma *Greedy* untuk menyelesaikan persoalan pengambilan keputusan yang kompleks dan dinamis secara *real-time.*
2. Desain Fungsi Heuristik : Merancang fungsi seleksi heuristik yang efektif untuk mengoptimalkan navigasi agen, manajemen sumber daya cat, dan prioritas target serangan.
3. Optimasi Komputasi : Melatih kemampuan untuk menulis kode Java yang optimal dan efisien agar mampu beroperasi di bawah batasan *bytecode* dan memori yang sangat ketat.
4. Koordinasi Agen Terdistribusi : Mengembangkan strategi komunikasi dan koordinasi antara agen yang terdesentralisasi menggunakan sistem pesan terbatas (4 *byte*) dan penanda visual (*markers*).

1.3 Aturan Menang dan Tiebreaker

Permainan dapat dimenangkan melalui beberapa kondisi utama :

1. Dominasi Wilayah : Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta dinyatakan sebagai pemenang instan.
2. Eliminasi total : Menghancurkan seluruh unit robot dan menara milik tim lawan.

Apabila hingga batas maksimal 2000 ronde tidak ada tim yang memenuhi kondisi di atas, maka pemenang akan ditentukan melalui mekanisme *tiebreaker* dengan urutan prioritas sebagai berikut:

1. Luas Area, total luas wilayah yang telah diwarnai oleh tim di peta.
2. Menara hidup, jumlah menara sekutu yang masih aktif di lapangan.
3. Kekayaan Chips, total akumulasi uang (*chips*) yang dimiliki tim.
4. Cadangan Paint, total akumulasi cat (*paint*) yang tersimpan di seluruh robot dan menara.
5. Jumlah Robot, total populasi robot sekutu yang masih bertahan.
6. Random, jika seluruh kriteria di atas tetap seri, pemenang dipilih secara acak.

BAB II : Landasan Teori

2.1 Dasar Teori Algoritma Greedy

Algoritma *Greedy* merupakan salah satu paradigma pemecahan masalah yang bekerja dengan cara membangun solusi langkah demi langkah secara bertahap. Prinsip utama dari algoritma ini adalah pada setiap langkahnya, keputusan yang diambil merupakan pilihan yang memberikan keuntungan paling besar atau hasil yang paling optimal pada saat itu juga, tanpa mempertimbangkan konsekuensi jangka panjang atau dampak pilihan tersebut di masa depan. Keputusan ini dikenal dengan istilah *local optimum* atau optimum lokal. Harapan utama dari penerapan strategi ini adalah bahwa rangkaian pilihan optimal di tingkat lokal tersebut pada akhirnya akan mengarah pada solusi optimal di tingkat global. Meskipun strategi ini tidak selalu menjamin solusi terbaik secara mutlak untuk semua jenis persoalan, algoritma *Greedy* seringkali memberikan hasil yang cukup mendekati optimal dengan waktu eksekusi yang jauh lebih cepat dibandingkan metode pencarian solusi lainnya.

Secara formal, terdapat lima elemen utama yang menyusun algoritma *Greedy.* Elemen pertama adalah himpunan kandidat yang berisi sekumpulan elemen mentah yang mungkin untuk membentuk sebuah solusi. Elemen kedua adalah himpunan solusi yang berfungsi sebagai tempat penampungan bagi kandidat-kandidat yang telah terpilih pada setiap tahapan. Ketiga adalah fungsi seleksi yang menjadi inti dari strategi ini, di mana algoritma menggunakan kriteria heuristik tertentu untuk kelayakan yang bertugas untuk memastikan bahwa kandidat yang dipilih tidak melanggar batasan-batasan (*constraints*) dari persoalan yang dihadapi. Terakhir adalah fungsi objektif yang berperan sebagai parameter penilaian untuk memaksimalisasi atau meminimalisasi hasil dari solusi yang telah dibangun.

2.2 Cara Kerja Program Battlecode Secara Umum

Program Battlecode beroperasi sebagai sebuah sistem multi-agen otonom di mana setiap unit robot dan menara mengeksekusi logika kendalinya sendiri secara independen. Setiap agen berjalan di dalam *Java Virtual Machine* (JVM) yang terpisah, sehingga koordinasi antar unit tidak dapat dilakukan melalui penggunaan variabel statis global, melainkan harus melalui mekanisme komunikasi pesan radio yang terbatas atau penempatan penanda visual di medan tempur. Permainan ini berjalan berbasis giliran yang dibagi ke dalam ronde-ronde. Dalam setiap gilirannya, sebuah unit akan menjalankan siklus rutin yang dimulai dengan fase pengindraan atau sensing. Pada fase ini, unit menggunakan berbagai fungsi API untuk mengumpulkan data mengenai lingkungan sekitar dalam radius pandang yang terbatas, seperti posisi robot kawan maupun lawan, kondisi pewarnaan lantai, serta keberadaan fitur peta seperti tembok atau reruntuhan menara.

Setelah data lingkungan terkumpul, unit akan masuk ke tahap pengambilan keputusan yang mengimplementasikan strategi algoritma tertentu, seperti algoritma *Greedy*. Unit akan mengevaluasi berbagai kemungkinan aksi yang dapat dilakukan, kemudian memilih satu aksi yang dinilai paling menguntungkan berdasarkan fungsi heuristik yang telah diprogramkan. Aksi fisik yang dilakukan dapat berupa pergerakan posisi, melakukan serangan untuk mewarnai lantai atau merusak lawan, hingga membangun infrastruktur baru. Pelaksanaan aksi ini sangat dibatasi oleh nilai cooldown serta ketersediaan sumber daya cat dan uang. Selain itu, efisiensi algoritma menjadi aspek yang sangat krusial karena adanya batasan bytecode limit. Jika logika pemrosesan

yang dijalankan unit terlalu rumit dan melebihi batas sumber daya komputasi tersebut, sistem akan menghentikan eksekusi kode unit tersebut secara paksa dan agen akan kehilangan kesempatan untuk beraksi pada ronde tersebut.

BAB III : Aplikasi Strategi Greedy

3.1 Mapping Persoalan Battlecode ke Elemen Greedy

Dalam ekosistem Battlecode, permainan berjalan secara dinamis setiap giliran (turn). Algoritma Greedy tidak secara tradisional digunakan untuk menyelesaikan seluruh permainan dalam satu hitungan matematis, melainkan dieksekusi pada level Micro-Decision (keputusan aksi per ronde) oleh masing-masing robot secara individual. Secara teknis, penyelesaian aksi setiap robot pada titik waktu tertentu dipetakan ke dalam enam elemen dasar algoritma greedy sebagai berikut:

3.1.1 Himpunan Kandidat

Merupakan kumpulan elemen mentah atau daftar kemungkinan langkah/target yang tersedia bagi robot pada ronde tersebut setelah memanggil sensor lingkungan dari memori API Engine. Tergantung tipe unitnya, himpunan kandidat ini meliputi:

- Kandidat arah pergerakan yang berisikan 8 titik mata angin (`Direction[] ALL_DIRS`).
- Kandidat target serangan atau pengecatan berupa seluruh ubin (`MapLocation`) yang berada di dalam jangkauan radar efek sentral radius serang (misalnya distSq <= 4 untuk Splasher).
- Kandidat target pembangunan konstruksi yang berisikan daftar array koordinat reruntuhan (`nearbyRuins`) hasil tangkapan sensor penglihatan unit Soldier.
- Kandidat produksi robot (spesifik untuk kelas Tower), yaitu pemilihan Blueprint tipe unit yang akan ditelurkan (misal: `UnitType.SPLASHER` atau `UnitType.SOLDIER`).

3.1.2 Himpunan Solusi

Himpunan Solusi merepresentasikan rekam jejak aksi spesifik atau parameter final yang dipilih oleh robot untuk dieksekusi pada giliran yang relevan. Himpunan ini hanya memuat satu aksi definitif pada satu titik waktu, seperti:

- Keputusan "Tembak/Mengecat" di `MapLocation (X, Y).`
- Keputusan "Bergerak" menuju arah `Direction.NORTH.`
- Keputusan "Menyelesaikan Konstruksi Pola" pada tipe Money Tower.
- Keputusan pergeseran prioritas paksa (interupsi), seperti mutlak pulang ke menara suplai cat.

3.1.3 Fungsi Kelayakan

Fungsi kelayakan adalah serangkaian predikat boolean yang menyaring kandidat untuk memastikan bahwa langkah tersebut legal, aman, dan dapat divalidasi oleh batasan sistem sebelum robot bertindak. Aturan-aturan kelayakan yang kami terapkan antara lain:

- Batas Waktu (Cooldown): Memastikan eksekutor bebas dari batasan jeda melalui validasi rc.isMovementReady() atau rc.isActionReady().
- Ketersediaan Logistik & Toleransi Keamanan: Mengevaluasi ambang batas cat. Jika cadangannya menembus angka krisis (misalnya di bawah 40 muatan untuk unit serbu Snowball atau 20% bagi unit spesialis), maka segala tindakan eksplorasi dan pertempuran dicoret demi mengutamakan Mode Refill pelarian.
- Kebebasan Jalur : Pada pergerakan arah, ditinjau apakah tujuan tersebut pas untuk berpijak (`rc.canMove(dir)`). Untuk konstruksi Soldier, sebuah

reruntuhan hanya layak (layak dikerjakan) jika dipastikan tidak ada rekan kerja (Soldier sekutu) lain yang sedang mengerjakannya dalam jarak 2 kuadratik.

3.1.4 Fungsi Seleksi

Fungsi Seleksi adalah mesin penimbang utama yang akan menyeleksi sisa kandidat pasca-penyaringan fungsi kelayakan. Masing-masing unit memilih himpunan dengan "skor/nilai" lokal paling menggiurkan (Greedy) pada suatu titik. Fungsi seleksi utama dari arsitektur bot yang kami racik adalah:

- Heuristik Inversi Jarak & Momentum Bergerak: Mengakumulasikan densitas nilai ubin belum diwarnai/area lawan ($1 / \sqrt{jarak}$) dirangkum bersandingan dengan tarikan bias masif ke arah pertengahan lokasi spawn lawan (Symmetry Pull) untuk menyeimbangkan penjelajahan dengan efisiensi area.
- Modifikasi Adjacency Penalty: Potongan skor diskon ekstrem (-50 poin) dikenakan ke setiap langkah yang mengacu tegak lurus menempel dengan unit sekutu, difungsikan serakah untuk memecah formasi berkumpul agar area pengecatan menyebar rapi layaknya jaring laba-laba.
- Heuristik AoE Bias Ekstrem : Menjatuhkan bom air tidak lagi sekadar menyasar zona nihil; jika target yang ditidik kebetulan saling berhimpit dengan koordinat Ruin yang sedang dicoba disempurnakan oleh Soldier, bobot serangan diberikan nilai bias ekstrem (+100 poin) agar memancing Splasher tersebut untuk instingtif bekerja mengawal pekerja.
- Spawning Threshold Menara: Secara rakus menimbang kas uang menara; di mana produksi langsung dicekik bila terdengar lalu lintas transmisi pesan bitmask (`MSG_HOLD_SPAWN`) dari para kuli hingga dana tabungan menembus >1000 chips.

3.1.5 Fungsi Objektif

Parameter matematis yang dirancang untuk dieksekusi sebagai bahan optimasi pada saat pemilihan di Fungsi Seleksi. Setiap perhitungan unit mengejar lokalisasi optimumnya sendiri, dengan tujuan optimasi sebagai berikut:

- Memaksimalkan (Maximize): Jumlah petak lawan dan kosong yang ditimpa dalam satu letusan bom cairan dengan membedakan radius dampak yang mematikan hingga bias sapuan ubin terluarnya. Memaksimalkan akumulasi properti pergerakan menembus relung titik cermin asal sasaran menara lawan.
- Meminimalkan (Minimize): Jarak kembali ke menara pengisian suplai demi efisiensi penyelesaian cooldown pasokan, meminimalkan sumbatan konduksi jalan sempit .

3.1.6 Fungsi Solusi

Fungsi akhir ini mengeksekusi parameter jawaban di kala pencarian konklusif per satu giliran usai dijamin absah, direpresentasikan melalui instruksi seruan `Clock.yield()`. Pemanggilan terminasi tunggal ini mendeklarasikan bahwa optimum titik pencarian telah tergali seratus persen pada ronde tersebut, memberikan waktu bernapas bagi yang mengembalikan estafet operasional robot menuju antrian putaran robot berikutnya dalam arena tempur. Secara global/level agregat makro, status penyelesaian sejati hanyalah konfirmasi mutlak bahwa persentase 70% cat di ubin

penjuru peta telah dibentangkan atau ketika total sisa musuh menyentuh angka mutlak nol.

3.2 Eksplorasi 3 Alternatif Solusi Greedy

3.2.1 mainbot

Pada rancangan main bot ini, kami menerapkan strategi yang kami sebut sebagai *Balanced Strategic Swarm*. Filosofi utama dari bot ini adalah menciptakan sistem multi-agen yang saling mendukung, di mana setiap unit memiliki peran spesifik yang berkontribusi pada stabilitas ekonomi dan penguasaan wilayah. Kami menyusun hirarki prioritas aksi yang ketat agar keputusan greedy yang diambil setiap unit memberikan keuntungan maksimal bagi tim secara kolektif. Berikut adalah rincian heuristik yang kami gunakan untuk setiap unit:

Untuk unit Menara (*Tower*), kami mengimplementasikan Heuristik Rasio Populasi untuk mengatur fungsi seleksi produksinya. Kami tidak ingin menara memproduksi robot secara acak, sehingga kami menetapkan target komposisi pasukan yang seimbang: 30% Soldier sebagai pembangun, 50% Splasher sebagai pelapis wilayah, dan 20% Mopper sebagai unit pertahanan. Sementara itu, untuk fungsi serangan, menara menggunakan Heuristik Nearest Enemy, di mana menara akan secara rakus menembak musuh yang memiliki jarak kuadrat (distance squared) terkecil guna menjaga keamanan zona spawn agar tetap steril dari gangguan lawan.

Pada unit *Soldier*, kami memfungsikan mereka sebagai unit infrastruktur utama dengan menggunakan Heuristik *Global Pattern Completion* sebagai prioritas tertinggi. Setiap ronde, Soldier kami akan memindai seluruh reruntuhan yang terlihat di radar. Jika ditemukan pola menara yang hampir selesai, mereka akan memprioritaskan untuk memberikan tembakan cat terakhir guna menghidupkan menara tersebut, karena kami menganggap penambahan titik spawn baru lebih menguntungkan daripada sekadar menyerang robot musuh. Jika tidak ada pembangunan, mereka menggunakan Heuristik *Safe Ruin Proximity* untuk mencari reruntuhan terdekat yang aman dari jangkauan Splasher musuh. Apabila tidak ada reruntuhan terdeteksi, kami menerapkan Heuristik Eksplorasi Kuadran berbasis ID robot ($ID \pmod 4$), yang memaksa setiap Soldier menyebar secara merata ke seluruh penjuru peta.

Untuk unit *Splasher*, fokus utama kami adalah optimalisasi penggunaan cat yang mahal melalui Heuristik Evaluasi Skor AoE. Sebelum mengeksekusi serangan, robot akan melakukan simulasi untuk menghitung berapa banyak petak kosong atau cat musuh yang dapat ditimpa di dalam radius ledakannya. Kami memberikan bobot yang dibobot yang lebih tinggi jika ledakan mengenai area inti lawan. Dalam aspek pergerakan, kami menerapkan Heuristik Kepadatan Inversi Jarak (*Inverse Distance Density*), di mana robot mengevaluasi arah mata angin berdasarkan akumulasi skor petak kosong yang dibobot dengan rumus $1 / \sqrt{jarak}$. Heuristik ini membuat *Splasher* kami secara alami tertarik pada "gumpalan" area kosong yang luas dibandingkan petak kosong yang terisolasi.

Unit *Mopper* kami desain sebagai unit interseptor taktis dengan Heuristik *Leashing* (Tali Pengikat). Kami membatasi radius pergerakan *Mopper* agar tetap berada dalam jangkauan 11 petak dari menara terdekat untuk memastikan pertahanan base tetap terjaga. Untuk fungsi seleksi target serangan, kami menerapkan Heuristik *High-Threat*

Targeting, di mana Mopper akan secara agresif memprioritaskan pengejaran terhadap *Splasher* musuh guna menyedot cadangan cat mereka. Kami memilih strategi ini karena melumpuhkan satu *Splasher* lawan secara signifikan akan mengurangi risiko wilayah kami dibom oleh serangan area lawan.

Terakhir, untuk seluruh unit robot, kami merancang Heuristik Navigasi `Bug0-Fuzzy`. Robot akan mencoba bergerak langsung ke arah target, namun jika terblokir oleh teman atau hambatan peta, fungsi seleksi pergerakan akan secara *greedy* mencoba arah rotasi terdekat (45 hingga 90 derajat) secara deterministik untuk menghindari kemacetan. Kami juga menerapkan Heuristik Manajemen Sumber Daya berbasis ambang batas kritis (30-50%). Melalui heuristik ini, setiap unit akan secara rakus memprioritaskan keselamatan diri untuk kembali ke menara sekutu saat cadangan cat menipis, guna memastikan bahwa sumber daya chips yang telah kami investasikan untuk membangun unit tersebut tidak hilang sia-sia di tengah medan tempur.

3.2.2 Snowball bot

Pada rancangan snowball Bot ini, kami menerapkan strategi yang kami sebut sebagai Distributed Heuristic Swarm (Kawanan Heuristik Terdistribusi). Filosofi utama dari bot ini adalah menciptakan sistem multi-agen yang bertindak secara greedy pada skala lokal namun mampu menghasilkan perilaku cerdas secara kolektif. Fokus utama kami adalah pada ekspansi agresif dan penguasaan reruntuhan (ruin) melalui hierarki prioritas aksi yang terstruktur. Hal ini memungkinkan setiap unit mengambil keputusan optimal secara mandiri tanpa memerlukan micro-management terpusat yang boros bytecode. Berikut adalah rincian heuristik yang kami terapkan pada setiap unit:

Untuk unit Menara (Tower), kami mengimplementasikan Heuristik Ekspansi Ekonomi Terstruktur dan Heuristik Rasio Populasi. Kami mengatur fungsi seleksi produksi dengan rasio dinamis: pada fase early game, kami memprioritaskan satu Splasher diikuti dua Soldier, sedangkan pada fase mid-to-late game, komposisi diseimbangkan menjadi rasio 1:1. Kami menetapkan urutan pembangunan menara secara ketat menggunakan prinsip Resource-First (Money, Paint, Money, Paint) guna menjamin percepatan eksponensial dalam suplai sumber daya. Selain itu, kami menerapkan fungsi Greedy Spawn-Throttle (Hold-Spawn) melalui sistem pesan; menara akan secara rakus menabung hingga 1000 chips dan menghentikan produksi robot jika menerima sinyal dari Soldier bahwa sebuah reruntuhan sedang dalam proses pembangunan. Menara juga dibekali Greedy Upgrade untuk segera naik ke level 3 setiap kali dana mencukupi.

Pada unit Soldier, kami memfungsikannya sebagai teknisi infrastruktur utama dengan Heuristik Exclusive Ruin Claiming sebagai prioritas tertinggi. Soldier akan memindai reruntuhan terdekat menggunakan Min-Dist Greedy, namun dengan syarat ketat: jika terdeteksi rekan sekutu dalam radius 2 petak dari reruntuhan tersebut, Soldier akan secara greedy meninggalkannya untuk mencari target lain. Hal ini memastikan pembagian kerja paralel dan mencegah penumpukan unit di satu titik. Saat berada di lokasi, Soldier mengeksekusi urutan aksi: Instant-Complete Pattern, Mark Pattern, lalu menembak petak terdekat. Kami juga menyertakan manuver Active Orbiting, di mana unit akan berputar 90 derajat mengelilingi bangunan jika terhalang rintangan atau berada di luar jangkauan tembak.

Untuk unit Splasher, fokus utama kami adalah efisiensi pengecatan dan perlindungan proses pembangunan melalui Heuristik Skor Area of Effect (AoE) Berbias Ekstrem. Sebelum menembak, robot melakukan simulasi matematis untuk menghitung akumulasi yield berdasarkan jumlah petak kosong atau cat musuh yang terdampak. Kami menyuntikkan bobot drastis sebesar +100.0 poin secara greedy jika titik ledak berada tepat pada reruntuhan yang sedang dibangun oleh sekutu. Heuristik ini membuat Splasher secara instingtif mendatangi reruntuhan dan bekerja sebagai pembersih jalur guna menyapu cat musuh yang menghalangi pola bangunan (Ruin Pattern).

Sebagai kerangka navigasi global, seluruh unit menggunakan Heuristik Pergerakan Vektor Multi-Variabel. Secara greedy, robot mengevaluasi 8 arah mata angin dan merangkum skor berdasarkan empat komponen: (1) Momentum untuk menjaga konsistensi arah, (2) Coverage berbasis invers jarak (1/√dist) untuk mengejar wilayah yang belum terpetakan, (3) Symmetry Pull untuk menarik unit ke arah titik cerminan peta, dan (4) Universal Adjacency Penalty. Variabel penalti ini memberikan diskon skor tajam (-50 poin) jika suatu langkah membuat robot berdiri bersebelahan dengan rekan sendiri, memaksa barisan unit kami menyebar luas seperti jaring laba-laba.

3.2.3 Splasher specialist bot

Pada rancangan Splasher specialist ini, kami menerapkan strategi yang kami sebut sebagai Splasher Specialist Territorial Expansion. Filosofi utama dari bot ini berfokus pada spesialisasi peran mutlak, di mana menara secara eksklusif hanya memproduksi unit Splasher. Tujuannya adalah penguasaan wilayah (territorial coverage) yang masif dan efisien melalui serangan area (AoE). Pembagian fungsi objektif dan pengambilan keputusan greedy-nya diuraikan sebagai: Pada unit Menara (Tower), fungsi seleksi produksi tidak didasarkan pada hirarki rasio yang rumit. Menara menggunakan Heuristik Produksi Deterministik Acak di mana ia mengekstrak satu arah putaran dan berusaha memproduksi unit Splasher secepat mungkin ke arah tersebut secara persisten.

Untuk unit Splasher, terdapat dua kendali krusial yang berhubungan dengan ketersediaan sumber daya:

A. Heuristik Opportunistic Refill: Apabila unit berada tepat di samping menara sekutu dan kondisi cat berada di bawah 90%, unit akan secara greedy menarik cat secara instan tanpa perlu beralih ke mode navigasi pulang.
B. Prioritas Refill-Constraint: Apabila sisa cat berada pada posisi kritis (< 20%) atau tidak cukup untuk melancarkan satu serangan radius (biaya 50 cat), robot akan membatalkan segala prioritas serangan dan secara serakah mencari rute untuk ditarik pulang ke menara suplai yang dicatat di kompas radar internal `nearestPaintTower`.

Dalam mengeksekusi serangannya, Splasher ini menerapkan Heuristik Dynamic AoE Yield. Pada saat merakit himpunan kandidat serangan, unit akan mensimulasikan tembakan pada semua ubin dalam radius serang jarak kuadrat 4. Unit ini menghitung bobot per ubin, di mana ubin netral di dalam seluruh area pelibatan akan terus diakumulasi nilainya, dan ubin milik musuh hanya akan diakumulasi jika berada di inti ledakan kerena hanya jarak ini yang dapat menimpa warna musuh. Setelah titik skor tertinggi ditemukan, serangan tidak serta merta dieksekusi, melainkan harus lolos dari pengawalan Ambang Batas Dinamis (Dynamic Threshold): skor_maks * 40 >

cat_sekarang. Keputusan algoritmik ini memaksa robot yang kehabisan cat untuk menuntut ledakan dengan persentase hasil jauh lebih masif baru berani menembak.

Terakhir, untuk mekanisme pergerakan digunakan modifikasi Heuristik Inverse Distance Coverage yang dilengkapi penegak ambisi simetris. Unit akan meninjau sekitarnya. Jika jumlah petak target di area internal kurang dari 5 titik, robot mengabaikan kalkulasi medan berbobot, dan banting setir menggunakan Global Symmetry Ambition, yang memaksanya bergerak luru menuju arah refleksi terbalik (simetri) dari kordinat asalnya di peta layar untuk segera membedah basis tim musuh.

3.3 Analisis Efisiensi dan Efektivitas Teoritis

3.3.1 Main bot

Dalam mengerjakan tugas ini, kendala paling nyata yang kami hadapi adalah batasan bytecode sebesar 17.500 per ronde. Kami menyadari bahwa algoritma pencarian jalur yang ideal akan memakan terlalu banyak sumber daya jika dijalankan oleh puluhan robot secara bersamaan, apalagi dengan keterbatasan memori yang hanya 8 MB. Oleh karena itu, kami memutuskan untuk menjaga navigasi tetap sederhana namun cerdas dengan menggunakan pendekatan Bug0 dan Fuzzy Move. Efisiensi dari pilihan ini terletak pada kompleksitas waktunya yang bersifat *O(1)* per langkah, di mana robot hanya perlu mengevaluasi delapan petak di sekelilingnya. Dengan menghemat *bytecode* di sektor navigasi, kami memiliki sisa sumber daya yang lebih besar untuk dialokasikan pada algoritma seleksi target pertempuran yang lebih kompleks.

Dari sisi efektivitas, kami melakukan optimasi pada bagaimana robot mengolah informasi sensor. Kami menemukan bahwa memanggil fungsi sensing seperti rc.senseMapInfo() berkali-kali di dalam sebuah perulangan adalah "pembunuh" utama *bytecode*. Untuk mengakalinya, kami menerapkan teknik *Pre-filtering*. Robot cukup memanggil sensor lingkungan sekali di awal gilirannya, lalu menyimpan data petak-petak penting ke dalam sebuah *array* lokal yang lebih ringan. Secara teoritis, cara ini menjamin robot kami tidak akan mengalami *lag* atau berhenti di tengah jalan karena kehabisan tenaga komputasi. Kami juga menambahkan logika memori jangka pendek untuk menyimpan posisi sebelumnya agar robot tidak mengalami gerakan "jitter" atau bolak-balik di satu titik yang sama, yang seringkali menjadi kelemahan algoritma *greedy* sederhana.

Selain itu, kami menganalisis bahwa penggunaan Heuristik Skor Terbobot pada unit *Splasher* jauh lebih efektif daripada sekadar menembak petak kosong terdekat. Dengan memberikan bobot nilai lebih tinggi pada cat musuh (2.5) dibanding petak kosong (1.0), Splasher kami secara otomatis akan bergerak menuju area yang paling merugikan lawan. Penggunaan rumus Inversi Jarak ($1 / \sqrt{jarak}$) dalam fungsi navigasi juga memastikan robot lebih tertarik pada kumpulan petak kosong yang berada di dekatnya daripada yang letaknya jauh. Secara teoritis, kombinasi penghematan *bytecode* dan ketajaman evaluasi lapangan ini membuat bot kami memiliki stabilitas performa yang tinggi meskipun jumlah unit di lapangan sudah sangat padat.

3.3.2 Snowball bot

Tantangan terbesar dalam efektivitas sistem ini adalah batasan bytecode sebesar 17.500 instruksi per ronde. Kami menyadari bahwa algoritma pencarian jalur iteratif

seperti A* akan menyebabkan lonjakan kompleksitas yang berisiko timeout. Oleh karena itu, efisiensi komputasi kami gantungkan pada navigasi heuristik vektor deterministik dengan kompleksitas O(1) per iterasi. Dengan membatasi pengujian hanya pada 8 petak tetangga, kami menghemat bytecode secara signifikan, memberikan ruang bagi sistem untuk memproses logistik Bitmasking Protocol dalam komunikasi antar-unit.

Dari sisi pemrosesan sensor, kami menerapkan teknik Global Pre-filtering State. Informasi lingkungan diserap cukup satu kali pada awal setiap ronde dan disimpan dalam penampung statis global (nearbyTiles, nearbyRobots). Strategi ini memangkas biaya bytecode boros yang biasanya muncul akibat pemanggilan API sensor yang berulang di dalam blok if-else. Secara teoritis, pembendungan "lag" komputasi ini menjamin bot tetap responsif dalam batas waktu yang ditentukan.

Lebih jauh, penggunaan penalti negatif dan momentum berhasil mengatasi kelemahan navigasi greedy konvensional yang sering menyebabkan robot terjebak dalam gerakan maju-mundur (jitter). Kombinasi antara penghematan bytecode dengan heuristik penahan dorongan primitif (spawn-throttle) memastikan pertumbuhan Bot Snowball berlangsung secara eksponensial. Sistem ini tidak hanya efisien di balik layar, tetapi juga eksplosif dalam melakukan ekspansi dan memenangkan penguasaan wilayah di medan tempur.

3.3.3 Splasher specialist

Dari segi pencapaian komputasi, kendala bytecode limit adalah risiko vital bagi Splasher yang harus menghitung dan mensimulasikan efek area secara masif. Maka dari itu, kami merancang optimasi instruksional yang dikenal sebagai Sensing Filtering dini. Robot pada setiap rondegannya akan memanggil API pindaian (`senseNearbyMapInfos`) cukup sekali. Data diproses untuk menyeleksi ubin mana yang masih kosong atau yang milik musuh ke dalam array lokal primitif statik (`paintableTiles`). Dengan pemangkasan ini, algoritma yang mencari himpunan solusi ledakan terbaik tidak perlu mengeksekusi loop kosong pada ubin berisi menara atau rintangan. Ini memangkas iterasi overhead java virtual machine hingga lebih dari 40% dalam lautan titik eksekusi.

Dari aspek efektivitas medan, penggunaan Heuristik Dynamic Threshold menyeimbangkan kengerian agresi dengan rasio keberlanjutan hidup di arena pertempuran. Sebagai contoh ekstrem, Splasher biasa yang nekat menyumbangkan ledakkan terakhir pada secuil ubin akan terjebak tak bersenjata seketika lawan merapatkan barisan. Namun pada setup robot Alternatif 3, semakin tiris cat yang ia himpun, robot semakin mensyaratkan kepadatan jumlah petak kosong di tempat ia akan memercik agar serangan bermakna optimal secara rasional. Tambahan lagi, Opportunistic Refill berhasil menjadi alat gerak yang sangat efektif secara tersembunyi karena hal tersebut tidak mencederai turn limit serta instruksi navigasi asalnya.

BAB IV : Implementasi dan Pengujian

4.1 Implementasi

Dalam tahap implementasi, kami menerjemahkan strategi *greedy* yang telah dirancang ke dalam bahasa pemrograman Java. Kami mengembangkan tiga variasi bot yang masing-masing memiliki fokus heuristik yang berbeda untuk mengevaluasi efektivitas pengambilan keputusan dalam berbagai situasi.

4.1.1 mainBot

4.1.1.1 Fungsi Utama

Fungsi `run(RobotController rc)` merupakan pusat kendali utama atau "jantung" dari setiap instansi agen otonom yang beroperasi di arena *Battlecode* 2025. Secara teknis, fungsi ini bertindak sebagai titik masuk (*entry point*) yang dieksekusi secara otomatis sejak unit dimunculkan hingga permainan berakhir atau unit tersebut dihancurkan oleh lawan. Mengingat setiap tipe robot memiliki spesifikasi, biaya sumber daya, dan tugas strategis yang berbeda, kami merancang fungsi ini untuk bekerja sebagai sebuah *dispatcher* atau pengarah alur logika. Tugas utamanya adalah mengenali identitas unit tersebut dan mengeksekusi sub-prosedur strategi *greedy* yang telah disesuaikan dengan peran unit masing-masing pada setiap ronde pertandingan.

Dalam implementasinya, fungsi ini membungkus seluruh logika keputusan di dalam sebuah perulangan tak terbatas (*infinite loop*). Pada setiap awal giliran, agen akan melakukan identifikasi tipe unitnya melalui fungsi API `rc.getType()`. Hasil identifikasi ini kemudian digunakan untuk memicu logika *greedy* spesifik, baik itu untuk Menara (*Tower*), *Soldier*, *Splasher*, maupun *Mopper*. Struktur ini kami pilih agar setiap agen dapat secara konsisten mengevaluasi kondisi lingkungan yang dinamis dan mengambil keputusan optimal lokal secara mandiri tanpa adanya ketergantungan variabel global antar unit.

Selain sebagai pengatur alur, fungsi ini juga kami rancang sebagai mekanisme pengaman operasional bot. Kami menggunakan blok penanganan kesalahan (*try-catch*) untuk memastikan bahwa jika terjadi pengecualian aksi (*GameActionException*) pada salah satu unit, program tidak akan berhenti secara keseluruhan. Di akhir setiap siklus perulangan, kami menempatkan perintah `Clock.yield()` yang berfungsi sebagai penanda berakhirnya tugas unit pada ronde tersebut. Hal ini sangat krusial dalam manajemen sumber daya komputasi agar bot kami tetap beroperasi di bawah ambang batas *bytecode limit* yang telah ditentukan sehingga setiap agen dapat terus berfungsi secara responsif sepanjang jalannya simulasi.

```
PROCEDURE run(RobotController rc):
   homeBase <- rc.getLocation()
   patrolDir <- PilihArahAcak(ALL_DIRS)

   WHILE True DO
       turnCount <- turnCount + 1
       TRY
           t <- rc.getType()
           IF t adalah Tower THEN
               runTower(rc)
```

```
            ELSE IF t == MOPPER THEN
                runMopper(rc)
            ELSE IF t == SOLDIER THEN
                runSoldier(rc)
            ELSE IF t == SPLASHER THEN
                runSplasher(rc)
        CATCH Exception
            // Abaikan error
        FINALLY
            Clock.yield() // Selesaikan giliran
```

4.1.1.2 runTower

Prosedur `runTower` kami rancang agar Menara dapat menjalankan dua fungsi utama secara bersamaan, yaitu sebagai fasilitas produksi unit dan sebagai garis pertahanan pasif. Dalam implementasinya, kami menggunakan pendekatan algoritma *greedy* yang dibagi menjadi dua tahap keputusan. Tahap pertama adalah fase pertahanan, di mana Menara akan mendeteksi seluruh robot musuh yang masuk ke dalam jangkauan sensor sebagai himpunan kandidat. Dari daftar musuh tersebut, kami menerapkan fungsi seleksi dengan memilih target yang memiliki jarak *Euclidean* paling dekat dari posisi Menara. Kami memilih kriteria jarak terdekat ini karena musuh tersebut merupakan ancaman yang paling nyata bagi keamanan zona produksi sehingga harus diprioritaskan untuk dihancurkan terlebih dahulu.

Tahap kedua adalah fase produksi unit, di mana Menara bertugas untuk memunculkan robot baru di lapangan. Untuk mencegah terjadinya ketimpangan jumlah pasukan, kami menggunakan fungsi seleksi yang diatur melalui sub-prosedur chooseSpawnType. Di sini, himpunan kandidatnya adalah tiga tipe robot utama (Soldier, Splasher, dan Mopper) serta delapan arah mata angin sebagai titik peletakannya. Kami menetapkan kriteria seleksi berdasarkan penjagaan rasio ideal pasukan, yaitu 30% Soldier, 50% Splasher, dan 20% Mopper. Robot yang akan diproduksi dipilih secara greedy berdasarkan tipe mana yang persentasenya saat itu masih berada di bawah ambang batas optimal tersebut.

Sebelum mengeksekusi tindakan menyerang maupun memproduksi unit, kami menerapkan fungsi kelayakan untuk memastikan efisiensi sumber daya. Kami melakukan pengecekan apakah target musuh benar-benar berada dalam radius serang Menara, serta memastikan bahwa petak yang dipilih untuk meletakkan robot baru tidak sedang ditempati unit lain dan tidak terhalang oleh fitur peta seperti tembok atau reruntuhan (*ruins*). Dengan menerapkan rangkaian logika ini, fungsi objektif yang ingin kami capai adalah menjaga kestabilan pertahanan di sekitar Menara sekaligus memaksimalkan laju jumlah unit yang dikirim ke area musuh untuk memenangkan penguasaan wilayah.

```
PROCEDURE runTower(rc):
   BacaPesanDariPasukan()
   IF BelumBisaBeraksi() RETURN
   // 1. Fase Serang (Pertahanan)
   KANDIDAT_MUSUH <- DapatkanSemuaMusuhDalamJarakSensor(rc)
```

```
   IF KANDIDAT_MUSUH tidak kosong THEN
       target_terdekat <- NULL
       jarak_minimum <- TAK_HINGGA
       FOR EACH musuh IN KANDIDAT_MUSUH:
           jarak <- JarakKuadrat(LokasiSekarang, musuh.Lokasi)
           IF jarak < jarak_minimum THEN
               jarak_minimum <- jarak
               target_terdekat <- musuh

                            IF    target_terdekat    !=    NULL    AND
BisaDiserang(target_terdekat.Lokasi) THEN
           Serang(target_terdekat.Lokasi)
           RETURN // Hanya bisa 1 aksi per giliran
   // 2. Fase Produksi Unit
   tipe_spawn <- chooseSpawnType()
   FOR EACH arah IN ALL_DIRS:
       lokasi_spawn <- LokasiSekarang + arah
       IF BisaMembangunRobot(tipe_spawn, lokasi_spawn) THEN
           BangunRobot(tipe_spawn, lokasi_spawn)
           CatatJumlahSpawn(tipe_spawn)
           RETURN
FUNCTION chooseSpawnType():
   total_robot <- moppersSpawned + splashersSpawned + soldiersSpawned
   IF total_robot == 0 THEN RETURN SOLDIER
   IF total_robot == 1 THEN RETURN MOPPER
   IF total_robot < 5  THEN RETURN SPLASHER

   // Penyeimbangan Rasio Jangka Panjang
   rasio_splasher <- splashersSpawned / total_robot
   rasio_mopper   <- moppersSpawned / total_robot
   rasio_soldier  <- soldiersSpawned / total_robot

   IF rasio_soldier  < 0.30 THEN RETURN SOLDIER
   IF rasio_splasher < 0.50 THEN RETURN SPLASHER
   IF rasio_mopper   < 0.20 THEN RETURN MOPPER

   RETURN SOLDIER
```

4.1.1.3 Soldier

Prosedur runSoldier kami rancang untuk unit yang memiliki peran ganda, yaitu sebagai kombatan sekaligus unit pekerja konstruksi. Tanggung jawab utama varian ini adalah melakukan ekspansi dengan cara merantai basis menara baru melalui pencarian reruntuhan (*ruins*) secara sistematis. Dalam mengimplementasikan logika ini, kami menerapkan paradigma *greedy* dengan hierarki prioritas aksi yang sangat ketat. Himpunan kandidat dalam persoalan ini mencakup reruntuhan yang belum terbangun, menara musuh, serta robot lawan yang berada dalam jangkauan radar. Fokus utama kami bukan sekadar membangun menara dari awal, melainkan melakukan *opportunistic build* atau mendahulukan penyelesaian konstruksi yang sudah hampir matang.

Fungsi seleksi yang kami gunakan memiliki dua tingkatan prioritas. Prioritas mutlak diberikan kepada reruntuhan yang hanya membutuhkan satu tembakan cat lagi

untuk selesai, karena secara *greedy* menghidupkan satu menara baru memberikan keuntungan strategis yang jauh lebih besar daripada aksi lainnya. Jika tidak ada pola yang hampir selesai, fungsi seleksi akan beralih mencari reruntuhan terdekat dari posisi unit saat ini, dengan preferensi tambahan pada reruntuhan yang berada di belahan peta yang sesuai dengan posisi awal tim kami (berdasarkan pembagian lebar peta per dua). Hal ini kami lakukan agar penyebaran menara tetap terorganisir dan tidak terlalu jauh merambah ke wilayah yang sulit dipertahankan di awal permainan.

Sebelum mengeksekusi pembangunan, kami menyertakan fungsi kelayakan sebagai langkah pengamanan sumber daya. Sebuah reruntuhan hanya layak dikerjakan apabila tidak terdeteksi adanya unit *Splasher* musuh di radius sekitarnya. Kami mempertimbangkan bahwa membangun menara di bawah ancaman serangan area musuh adalah tindakan yang tidak efisien karena menara tersebut akan hancur dalam waktu singkat. Selain itu, kami membatasi waktu pengerjaan melalui variabel ruinAttemptTurns. Jika proses pembangunan tertahan lebih dari 12 ronde, robot akan secara *greedy* membatalkan tugas tersebut dan mencari target lain. Dengan strategi ini, fungsi objektif yang ingin kami capai adalah maksimalisasi luas wilayah penguasaan melalui penempatan titik *spawn* dan ekonomi baru pada lokasi-lokasi yang paling optimal.

```
PROCEDURE runSoldier(rc):
   LokasiSekarang <- rc.getLocation()
   cat_sekarang   <- rc.getPaint()

   PerbaruiCacheMenaraSekutu()
   EstimasiLokasiMenaraMusuh()
   // Prioritas 0: Isi Ulang Cat
   IF cat_sekarang < 30% Kapasitas Maksimal THEN
       doRefill(rc, LokasiSekarang)
       RETURN

     //  Prioritas  1:  Selesaikan  Pola  Apapun  Yang  Hampir  Selesai  di
Sekitar
   IF tryFinishAnyPattern(rc, UbinSekitar, LokasiSekarang) == TRUE THEN
       tryPaintUnderfoot(rc)
       RETURN
   // Prioritas 2: Bangun Pola Baru
   isBuilder <- (rc.getID() mod 2 == 0) OR (soldiersSpawned <= 2)
   IF isBuilder THEN
       ruin_target <- findSafeRuinInSector(rc)
   ELSE
       ruin_target <- NULL

   IF ruin_target != NULL THEN
       IF ruin_target == currentRuin THEN
           ruinAttemptTurns <- ruinAttemptTurns + 1
       ELSE
           currentRuin <- ruin_target
           ruinAttemptTurns <- 0

       // Jika belum menyerah pada ruin ini
```

```
        IF ruinAttemptTurns < ABANDON_THRESHOLD (12 giliran) THEN
            IF buildAtRuin(rc, ruin_target) == TRUE THEN
                tryPaintUnderfoot(rc)
                RETURN
        ELSE
            currentRuin <- NULL // Menyerah dan cari target baru
            ruinAttemptTurns <- 0
   // Prioritas 3: Serang Menara Musuh
   sudah_menyerang <- FALSE
   IF MenaraMusuhDitemukan AND BisaMenyerang THEN
        IF Jarak(MenaraMusuh) <= RadiusSerang THEN
            Serang(MenaraMusuh)
            sudah_menyerang <- TRUE
   // Prioritas 4: Serang Robot Musuh Terdekat
   IF NOT sudah_menyerang AND BisaMenyerang THEN
        musuh <- nearestEnemy(rc)
        IF musuh != NULL AND BisaDiserang(musuh.Lokasi) THEN
            Serang(musuh.Lokasi)
            sudah_menyerang <- TRUE
   IF NOT sudah_menyerang THEN tryPaintUnderfoot(rc)
   // Logika Navigasi / Pergerakan
   IF rc.isMovementReady() THEN
         IF currentRuin != NULL AND ruinAttemptTurns < ABANDON_THRESHOLD
THEN
            moveFuzzy(rc, ArahKe(currentRuin))
              ELSE IF MenaraMusuhDitemukan AND JarakKe(MenaraMusuh) <
JarakKe(HomeBase) THEN
            moveFuzzy(rc, ArahKe(MenaraMusuh))
        ELSE
            // Eksplorasi Kuadran Peta berdasarkan ID
            kuadran <- rc.getID() mod 4
            targetEksplorasi <- HitungTitikPusatKuadran(kuadran)
            moveFuzzy(rc, ArahKe(targetEksplorasi))
```

4.1.1.4 Splasher

Prosedur runSplasher kami rancang khusus untuk mengelola unit yang berfungsi sebagai alat kontrol matriks melalui serangan area (*Area-of-Effect*). Kami menyadari bahwa serangan unit ini sangat mahal karena memakan biaya 50 unit cat untuk satu kali tembakan sehingga kami menerapkan algoritma *greedy* agar setiap serangan memberikan hasil yang sepadan. Dalam proses pengambilan keputusan, himpunan kandidat yang kami evaluasi mencakup seluruh ubin yang berada dalam jangkauan radar serangan, yaitu dengan nilai *distance squared* maksimal 4. Dari sekian banyak kandidat tersebut, robot harus menentukan satu titik fokus sasaran yang paling menguntungkan sebagai himpunan solusi.

Fungsi seleksi yang kami gunakan bekerja dengan cara mensimulasikan dampak ledakan di setiap titik kandidat. Robot akan menghitung seberapa banyak ubin kosong atau ubin milik musuh yang bisa terkena percikan warna tim kami. Kami menetapkan heuristik di mana penimpaan (*overwrite*) cat musuh pada radius inti (distSq <= 2)

dihargai sama mahalnya dengan mewarnai ubin kosong. Dengan cara ini, robot akan secara rakus memilih area yang memiliki kepadatan warna "asing" tertinggi agar konversi warna menjadi milik tim kami terjadi secara masif dalam satu kali aksi.

Sebelum tindakan serangan dieksekusi, kami menyertakan fungsi kelayakan untuk memastikan efisiensi tindakan tersebut. Robot akan memeriksa apakah cadangan cat yang dimiliki masih mencukupi ambang batas minimal 50 unit, serta memastikan bahwa titik sasaran tembak tidak terhalang oleh unit lawan yang dapat mengurangi efektivitas sebaran warna. Melalui strategi ini, fungsi objektif yang ingin kami capai adalah memaksimalkan penambahan metrik persentase penguasaan wilayah tim dengan frekuensi tembakan yang seefisien mungkin sehingga sumber daya cat yang terbatas tidak terbuang sia-sia pada area yang sudah berwarna searah.

```
PROCEDURE runSplasher(rc):
   LokasiSekarang <- rc.getLocation()
   cat_sekarang   <- rc.getPaint()

   PerbaruiCacheMenaraSekutu()
   // Prioritas 0: Isi Ulang Taktis & Darurat
   IF MenaraTerdekat != NULL AND cat_sekarang < 70% Kapasitas
      AND JarakKuadrat(MenaraTerdekat) <= 4 AND BisaAksi THEN
       LakukanTransferCat(MenaraTerdekat)

    IF cat_sekarang < 30% Kapasitas OR cat_sekarang < BiayaSerang (50)
THEN
       doRefill(rc, LokasiSekarang)
       RETURN
   // Prioritas 1: Serangan Area Berbasis Greedy
   IF BisaBeresaksi AND cat_sekarang >= BiayaSerang THEN
       KANDIDAT_SERANG <- Semua lokasi dalam radius serangan (distSq <=
4)
       skor_tertinggi <- 0
       target_terbaik <- NULL

       FOR EACH target IN KANDIDAT_SERANG:
           skor <- 0
           FOR EACH ubin_dalam_radius_cipratan(target):
               IF ubin.cat == KOSONG THEN
                   skor <- skor + 1
               ELSE IF ubin.cat == MUSUH AND JarakKuadrat(target, ubin)
<= 2 THEN
                    skor <- skor + 1 // Area inti cipratan bisa menimpa
cat musuh

           IF skor > skor_tertinggi THEN
               skor_tertinggi <- skor
               target_terbaik <- target

       IF target_terbaik != NULL THEN
           Serang(target_terbaik)
   // Prioritas 2: Navigasi
   IF rc.isMovementReady() THEN
       splasherNavigate(rc, UbinSekitar, LokasiSekarang)
```

```
FUNCTION splasherNavigate(rc):
   jumlah_menarik <- HitungUbin(Kosong OR Musuh) disekitar
   LuasPeta <- Lebar * Tinggi

   // Heuristik Jarak/Visibilitas (Jangan gunakan di peta kecil)
   IF jumlah_menarik >= 3 AND LuasPeta >= 900 THEN
       arah_terbaik <- TENGAH
       skor_maks <- -1.0

       FOR EACH arah IN ALL_DIRS:
           lokasi_depan <- LokasiSekarang + arah
                  skor_arah  <-  HitungSkorInverseDistance(lokasi_depan,
UbinMenarik)
           IF skor_arah > skor_maks THEN
               skor_maks <- skor_arah
               arah_terbaik <- arah

       IF arah_terbaik != TENGAH THEN
           moveFuzzy(rc, arah_terbaik)
           RETURN

   // Tujuan Global: Refleksi Rotasional dari Home Base
    tujuan_global <- Titik(LebarPeta - 1 - HomeBase.X, TinggiPeta - 1 -
HomeBase.Y)
   IF MenaraMusuhDitemukan THEN tujuan_global <- MenaraMusuh
   moveFuzzy(rc, ArahKe(tujuan_global))
```

4.1.1.5 Mopper

Prosedur runMopper kami rancang untuk mengelola unit yang berfungsi sebagai mesin tempur defensif sekaligus pasukan anti-huruhara di garis depan pertahanan tim. Berbeda dengan unit lain, *Mopper* difokuskan hanya untuk menargetkan unit lawan atau menghapus jejak cat musuh guna menjaga stabilitas wilayah. Dalam proses pengambilan keputusan *greedy*, unit ini mengevaluasi himpunan kandidat yang terdiri dari seluruh robot musuh yang terdeteksi, terutama unit *Splasher* yang dianggap sangat membahayakan keberadaan reruntuhan (*ruins*) kami, serta ubin-ubin di sekitar yang telah diwarnai oleh pihak lawan. Himpunan solusi yang dihasilkan mencakup eksekusi ayunan area (*Mop Swing*) untuk menghadapi musuh yang bergerombol, serangan langsung pada unit individu, maupun pembersihan lantai dari cat musuh.

Kami menerapkan fungsi seleksi yang secara spesifik memprioritaskan unit *Splasher* lawan dengan jarak Euclidean terdekat untuk segera dilumpuhkan. Jika tidak ditemukan unit *Splasher* di dalam jangkauan sensor, kriteria seleksi akan beralih secara rakus kepada jenis lawan apa pun yang posisinya paling dekat dengan lokasi agen saat itu. Pendekatan ini kami ambil agar ancaman terbesar dapat segera dihilangkan sebelum musuh sempat melakukan serangan area yang merugikan wilayah tim kami.

Untuk menjaga integritas pertahanan objek vital, kami menetapkan fungsi kelayakan yang cukup ketat melalui aturan *Leash Limit* atau batas tali pengikat. Agen

Mopper dilarang keras untuk bergerak maju menyerang atau mengejar target menjauhi perimeter aman jika jarak koordinat Euclidean terhadap menara pertahanan terdekat telah melampaui ambang batas yang ditentukan, yaitu lebih dari 130 satuan kuadrat API (*distance squared*). Melalui pembatasan gerak dan penajaman prioritas target ini, fungsi objektif yang ingin kami capai adalah menghindari penetrasi pasukan *Splasher* musuh ke area inti guna memastikan keberlanjutan ekspansi dan pembangunan infrastruktur tim kami tetap terjaga dengan aman.

```
PROCEDURE runMopper(rc):
   LokasiSekarang <- rc.getLocation()
   cat_sekarang   <- rc.getPaint()

   PerbaruiCacheMenaraSekutu()
   // Prioritas 0: Isi Ulang Cat
   IF cat_sekarang < 50% Kapasitas THEN
       doRefill(rc, LokasiSekarang)
       RETURN
     // Prioritas 1: Aturan Tali (Leash) - Jangan jauh dari menara
terdekat
      IF MenaraTerdekat != NULL AND JarakKuadrat(MenaraTerdekat) >
BATAS_LEASH (130) THEN
       moveFuzzy(rc, ArahKe(MenaraTerdekat))
       RETURN
   // Prioritas 2: Kunci Target Robot Musuh
   target_robot <- selectMopperTarget(rc)
   IF target_robot != NULL THEN
       engageMopperTarget(rc, target_robot)
       RETURN
   // Prioritas 3: Hapus Cat Musuh
   ubin_cat_musuh <- nearestVisibleEnemyPaint(rc)
   IF ubin_cat_musuh != NULL THEN
       IF BisaBeraksi AND BisaDiserang(ubin_cat_musuh) THEN
           Serang(ubin_cat_musuh) // Menghapus cat musuh
       IF BisaBergerak AND Jarak > 2 THEN
           moveFuzzy(rc, ArahKe(ubin_cat_musuh))
       RETURN
   // Prioritas 4: Patroli Garis Depan
   mopperPatrol(rc)
FUNCTION selectMopperTarget(rc):
   Prioritaskan Splasher Musuh terdekat
   Jika tidak ada, kembalikan robot musuh jenis lain terdekat.
FUNCTION engageMopperTarget(rc, target):
   // Gunakan ayunan area (MopSwing) jika musuh bergerombol
   cluster <- HitungMusuhSekitar(target.Lokasi)
   IF cluster >= 2 AND BisaMopSwing(ArahKeTarget) THEN
       MopSwing(ArahKeTarget)
   ELSE IF BisaDiserang(target.Lokasi) THEN
       Serang(target.Lokasi)
   ELSE IF BisaMopSwing(ArahKeTarget) THEN     // Ayunan darurat
       MopSwing(ArahKeTarget)

   IF BisaBergerak AND Jarak > 2 THEN
```

```
moveFuzzy(rc, ArahKe(target.Lokasi))
```

4.1.1.6 Helper Methods

4.1.1.6.1 findSafeRuinInSector(rc)

Fungsi `findSafeRuinInSector` kami rancang sebagai lapisan keamanan dalam pengambilan keputusan pembangunan infrastruktur oleh unit *Soldier*. Mengingat proses pembangunan menara membutuhkan waktu dan sumber daya yang besar, kami mengimplementasikan fungsi kelayakan (*feasibility check*) yang sangat ketat untuk meminimalisir risiko kegagalan. Dalam algoritma ini, himpunan kandidat terdiri dari seluruh reruntuhan (*ruins*) yang tertangkap oleh sensor unit. Sebelum masuk ke tahap penilaian skor, setiap kandidat harus melewati uji kelayakan: pertama, reruntuhan tidak boleh sedang ditempati oleh robot lain, dan kedua, tidak boleh ada unit *Splasher* musuh dalam radius 9 petak di sekitar lokasi tersebut. Kami menetapkan kriteria ini karena menara yang baru dibangun sangat rentan hancur seketika jika terkena satu serangan area musuh.

Setelah kandidat dinyatakan layak, fungsi seleksi akan melakukan pembobotan skor menggunakan variabel jarak dan kepemilikan sektor. Kami memberikan skor tambahan yang besar (10.000 poin) jika reruntuhan berada di belahan peta yang menjadi area operasional tim kami, kemudian dikurangi dengan jarak kuadrat ke lokasi tersebut. Heuristik ini bertujuan untuk membentuk rantai ekspansi menara yang terorganisir dari basis awal tim menuju ke depan. Hasil akhir dari fungsi ini adalah satu lokasi reruntuhan yang paling optimal secara lokal bagi *Soldier* untuk segera didatangi dan dibangun.

```
FUNCTION findSafeRuinInSector(rc):
   KANDIDAT <- Semua Ruin Yang Terlihat
   ruin_terbaik <- NULL
   skor_tertinggi <- NILAI_MINIMUM

   FOR EACH ruin IN KANDIDAT:
       IF AdaRobotDiAtas(ruin) THEN LanjutKeRuinBerikutnya

       // Pengecekan Keamanan Peledak
       IF AdaSplasherMusuhDalamRadius9(ruin.Lokasi) THEN
              LanjutKeRuinBerikutnya // Skip karena akan dirusak
lagi

         cek_sektor_saya <- Apakah Ruin Berada di Belahan Peta
Milik Tim Kita?

         // Jarak memengaruhi pemilihan untuk membentuk rantai
(chaining)
```

```
                skor  <-  (cek_sektor_saya  ?  10000  :  0)  -
JarakKuadrat(LokasiSaya, ruin.Lokasi)

       IF skor > skor_tertinggi THEN
           skor_tertinggi <- skor
           ruin_terbaik <- ruin

   RETURN ruin_terbaik
```

4.1.1.6.2 tryFinishAnyPattern(rc)

Fungsi `tryFinishAnyPattern` kami implementasikan sebagai instruksi *Greedy* dengan tingkat Prioritas Mutlak (*Absolute Priority* atau *Priority 0*). Fungsi ini bertujuan untuk mengatasi salah satu kelemahan umum algoritma *greedy* sederhana, yaitu kondisi di mana robot terlalu fokus mengejar target primer di kejauhan sehingga melewatkan peluang besar yang ada di depan mata. Dalam logika ini, setiap unit secara oportunistik akan memindai seluruh pola menara di sekitarnya. Jika ditemukan menara yang secara teknis sudah selesai polanya namun belum diaktifkan, atau menara yang hanya membutuhkan satu polesan cat terakhir, robot akan secara rakus melakukan "pindah jalur" atau interupsi tugas sementara untuk menyelesaikan pembangunan tersebut.

Keputusan ini kami anggap sebagai solusi paling optimal saat itu karena menghidupkan sebuah menara baru memberikan keuntungan strategis yang jauh lebih tinggi (seperti penambahan titik *spawn* dan suplai sumber daya) dibandingkan aksi standar lainnya. Jika fungsi ini menemukan ubin pola yang masih kosong, robot akan melakukan serangan cat dan secara bertahap bergerak mendekat melalui fungsi navigasi moveFuzzy. Dengan mengembalikan nilai *boolean* TRUE saat aksi diambil, kami memastikan bahwa robot segera mengakhiri gilirannya setelah melakukan kontribusi kritis pada infrastruktur tim, memastikan tidak ada sumber daya komputasi atau aksi yang terbuang sia-sia.

```
FUNCTION tryFinishAnyPattern(rc):
   FOR EACH ruin IN LahanRuin_Di_Jangkauan_Penglihatan DO:
       IF Engine_PolaSudahSelesai(ruin) == TRUE THEN
           PanggilFungsiPenyelesaianLengkap()
           MenyerahkanTargetRuinSebelumnya() // Clear target
           RETURN TRUE // Aksi Diambil, Giliran Selesai!

       // Cek bagian dari ruin tersebut mana yang perlu dicat
       FOR EACH ubin_di_sekitar_ruin IN Radius 8 DO:
                   IF ubin butuh dicat AND cat == KOSONG AND
BisaDiserang(ubin) THEN
               SerangCatTarget(ubin)
                moveFuzzy(rc, ArahMendekatiRuin) // Sambil ngesot
sedikit demi sedikit
               RETURN TRUE // Aksi Diambil

    RETURN FALSE // Tidak ada objek penyelesaian cepat di depan
mata
```

4.1.1.7 Navigation

Logika kerja algoritma ini kami analogikan seperti manusia yang sedang meraba dinding di dalam gua yang gelap. Prinsip utamanya adalah mencoba bergerak lurus ke arah target; namun, jika jalan di depan terhalang oleh tembok atau unit lain, agen tidak akan berhenti, melainkan mulai menyusuri tepi hambatan tersebut. Agen akan secara konsisten mengikuti satu arah rotasi (kanan terus atau kiri terus) sampai ia menemukan celah atau ruang kosong yang memungkinkan agen untuk kembali berorientasi menuju target aslinya.

Secara teknis, fungsi ini bekerja dalam tiga tahapan *greedy* lokal. Tahap pertama, fungsi akan selalu mencoba melakukan pergerakan lurus ke arah tujuan asli dan melakukan reset pada variabel pelacakan jika berhasil. Jika jalur lurus tidak layak karena terhalang, fungsi masuk ke tahap kedua, yaitu memulai mode pelacakan dinding (*wall tracing*). Kami menambahkan variabel trace_turns dengan ambang batas 15 giliran; jika agen merasa terjebak terlalu lama di satu sisi pelacakan, agen akan secara otomatis mengganti arah rotasi (dari kanan ke kiri atau sebaliknya) untuk memecah pola pergerakan yang statis. Pada tahap terakhir, fungsi seleksi akan mengevaluasi delapan kandidat arah mata angin secara melingkar (searah atau berlawanan jarum jam) dan mengeksekusi pergerakan pada arah pertama yang dinyatakan layak oleh sistem. Dengan pendekatan navigasi lokal ini, kami dapat memastikan robot tetap bergerak dinamis menuju target tanpa harus mengonsumsi *bytecode* yang berlebihan.

```
FUNCTION moveFuzzy(rc, ArahTujuan):
   IF TidakBisaBergerak OR ArahTujuan == TENGAH THEN RETURN

   // 1. Coba Jalan Lurus Dulu
   IF BisaBergerak(ArahTujuan) THEN
```

```
        Bergerak(ArahTujuan)
        trace_turns <- 0
        RETURN

    // 2. Terhalang Objek / Tembok, Lacak Dinding
    trace_turns <- trace_turns + 1
    IF trace_turns > 15 THEN
        GantiArahPelacakan(Kiri menjadi Kanan, atau sebaliknya)
        trace_turns <- 0

    // 3. Tes rotasi 8 arah searah/berlawanan jarum jam
    arah_sekarang <- ArahTujuan
    FOR i = 1 TO 7 DO
        arah_sekarang <- Putar(arah_sekarang, KeArahPelacakan)
        IF BisaBergerak(arah_sekarang) THEN
            Bergerak(arah_sekarang)
            RETURN
```

4.1.2 Snowball bot

4.1.2.1 Fungsi utama

`run(RobotController rc)` merupakan kerangka kerja sekaligus sakelar sentral yang berjalan setiap siklus giliran (turn). Pada tahapan eksekusi ini, unit kami difokuskan memanggil informasi alam di sekitarnya hanya satu kali per siklus (Global Single Sensing) demi menekan overhead operasional yang mahal. Setelah penyadapan memori selesai, agen disortir menurut tipenya. Satu kaidah Greedy yang berlaku statis lintas robot adalah pertimbangan kelangsungan hidup: Begitu unit berada di zona kritis level cat tertentu, prosedur `handleRefill` diaktifkan dan mengubah paksa keadaan eksekusi robot menjadi mode pelarian mutlak (*Strict Refill Flee Mode*).

```
PROCEDURE run(RobotController rc):
   CetakLogAwalKeKonsol("snowball Bot Starting...")

   WHILE True DO
       turnCount <- turnCount + 1
       TRY
             // Global Sensing: Simpan pandangan satu kali untuk
menghemat Bytecode
           LokasiSekarang     <- rc.getLocation()
           InfoLingkungan     <- rc.senseNearbyMapInfos()
           InfoRobot          <- rc.senseNearbyRobots()
           InfoReruntuhan     <- rc.senseNearbyRuins()

           PerbaruiLokasiPaintTowerTerdekat(rc)
            SetHomeLocPermanenSekali(rc) // Fallback rumah seumur
hidup
```

```
            tipe <- rc.getType()
            IF tipe adalah Menara (Tower) THEN
                runTower(rc)
            ELSE
                // Penanganan Logistik Anti-Kelumpuhan
                handleRefill(rc)
                readMessages(rc) // Sinkronisasi Intel Bitmasking

                IF isRefilling == TRUE THEN
                       // STRICT REFILL: Dilarang menyerang, paksa
unit pulang meminum cat
                          targetPulang <- PaintTowerTerdekat ATAU
homeLoc
                    IF BisaBergerak() THEN
                                  IF JarakKuadrat(LokasiSekarang,
targetPulang) > 2 THEN
                                     moveWithAdjacencyPenalty(rc,
ArahKe(targetPulang))
                        ELSE IF targetPulang == NULL THEN
                            navigateGreedy(rc, TRUE) // Flee Mode
                      CONTINUE // Abaikan instruksi perang, segera
lewati giliran ini

                // Mode Tempur dan Ekspansi Normal
                IF tipe == SPLASHER THEN
                    runSplasher(rc)
                ELSE IF tipe == SOLDIER THEN
                    runSoldier(rc)
        CATCH Exception
            CetakError()
        FINALLY
            Clock.yield() // Akhiri giliran komputasi saat ini
```

4.1.2.2 Tower

Prosedur runTower kami rancang agar Menara mampu menjadi poros pengatur keseimbangan mata uang secara pasif. Dalam implementasinya, kami menyajikan fase ekonomi adaptif Greedy Upgrade dan pembatasan pencetakan robot (Greedy Spawn Throttle). Menara mendengarkan gelombang pesan sekutu untuk menemukan MSG_HOLD_SPAWN (sinyal tahan spawn karena ada reruntuhan yang siap dibandung di medan tempur). Dari sana, menara akan membatasi pembelian pasukan dan memandu arah dompet ke tahap penahanan. Tahap berikutnya adalah memajukan populasi dengan heuristik Spawning, di mana Menara bertugas untuk memunculkan spesialis sesuai waktu siklus.

```
PROCEDURE runTower(rc):
   holdSpawnSignal <- FALSE
```

```
   // 1. Relai Pesan dan Pendeteksian Tahan Ekonomi
   PESAN_MASUK <- BacaSemuaPesanRadar(rc)
   FOR EACH pesan IN PESAN_MASUK:
       IF tipe_pesan == MSG_HOLD_SPAWN THEN
           holdSpawnSignal <- TRUE
       RelayPesanKeKawanSekitar(rc, pesan)

   // 2. Broadcast Lokasi Reruntuhan Ke Pasukan Baru Muncul
   ReruntuhanTerdekat <- AmbilReruntuhanBelumDibangun(rc)
   IF ReruntuhanTerdekat != NULL THEN
       KirimPesan(MSG_RUIN_FOUND, ReruntuhanTerdekat)

   IF BelumBisaBeraksi() RETURN

   // 3. Fase Greed Tumbuh Ke Atas (Upgrade)
   IF UangCukupDanBisaUpgrade() THEN
       UpgradeTower(LokasiSekarang)

   // 4. Pengendalian Keran Produksi (Spawn Throttle)
    // Tahan uang produksi hingga mencapai 1000 jika ditagih pembangunan
lapangan
   batas_spawn <- holdSpawnSignal ? 1000 : 250

   IF UangTim >= batas_spawn AND BisaBeraksi() THEN
       tipe_spawn <- decideSpawnType()
       FOR EACH arah IN ALL_DIRS:
           lokasi_spawn <- LokasiSekarang + arah
           IF BisaMembangunRobot(tipe_spawn, lokasi_spawn) THEN
               BangunRobot(tipe_spawn, lokasi_spawn)
               CatatJumlahStatistikSpawn(tipe_spawn)
               BREAK

FUNCTION decideSpawnType():
   total <- splasherSpawned + soldierSpawned
   // Heuristik Bukaan Peta: Penjelajah Pertama -> Pekerja -> Pekerja
   IF total == 0 THEN RETURN SPLASHER
   IF total == 1 OR total == 2 THEN RETURN SOLDIER

   // Mid-Late Game: Jaga paritas pekerja dan serdadu AoE di rasio 1:1
   IF soldierSpawned <= (1 * splasherSpawned) THEN
       RETURN SOLDIER
   ELSE
       RETURN SPLASHER

FUNCTION decideTowerToBuild(rc):
     // Ekstrem Ekonomi: Paksa penanaman modal Money dan Paint silih
berganti
   // Pola: Money -> Paint -> Money -> Paint -> Money -> Paint
   nt <- rc.getNumberTowers()
   IF nt <= 1 THEN RETURN LEVEL_ONE_MONEY_TOWER
   IF nt == 2 THEN RETURN LEVEL_ONE_PAINT_TOWER
   // Lanjut seterusnya, dan silih berganti Defense setelah tower ke 7
```

4.1.2.3 Soldier

Prosedur runSoldier kami desain untuk menjadikan varian ini sebagai fondasi pilar perluasan wilayah kami (arsitek). Kami membubuhi agen ini dengan paradigma fiksasi objektif ekstrem (Klaim Reruntuhan Eksklusif). Sebuah himpunan kandidat tidak dipilih mentah-mentah dari jarak *Euclidean* semata. Occupation Check akan menepis target yang sedang dikerjakan teman sendiri. Apabila Soldier sampai pada Ruin, mereka menerapkan hirarki eksekusi Opportunistic Pattern: dari Penyelesaian Langsung (Instant Build), Penandaan Cetak Biru (*Mark Pattern*), tembakan radius kecil, lalu jika terjebak, mereka mendemonstrasikan *Active Orbiting* dengan meroda mengitari sudut buntu reruntuhan.

```
PROCEDURE runSoldier(rc):
   // Prioritas 1: Asuransi Cat
   IF CatSekarang < 25 THEN
       KirimPesanMintaRefill(MSG_NEED_REFILL, LokasiSekarang)

   // Prioritas 2: Kuasai Reruntuhan Eksklusif
   IF targetRuin == NULL THEN
       jarak_minimum <- TAK_HINGGA
       FOR EACH ruin IN ReruntuhanRadiusPandang:
           IF SudahTerdapatTower(ruin) THEN LanjutkanPencarian

           // Pengecekan Okupansi: Satu Reruntuhan, Satu Kuli
                           IF  AdaTemanSoldierRadiusDuaDekat(ruin)  THEN
LanjutkanPencarian

           jarak <- JarakKuadrat(LokasiSekarang, ruin)
           IF jarak < jarak_minimum THEN
               jarak_minimum <- jarak
               targetRuin <- ruin

              IF  targetRuin  !=  NULL  THEN  KirimPesan(MSG_RUIN_FOUND,
targetRuin)
   ELSE IF SudahTerdapatTower(targetRuin) THEN
       targetRuin <- NULL // Lepas tugas karena sudah rampung

   // Prioritas 3: Eksekusi Konstruksi / Logika Active Orbiting
   IF targetRuin != NULL THEN
       tipeTower <- decideTowerToBuild(rc)
                            IF    BersebelahanDengan(targetRuin)    ATAU
PersisDiAtasTarget(targetRuin) THEN
           KirimSinyalSokonganUang(MSG_HOLD_SPAWN, targetRuin)

           // 1. Eksekusi Skema Spontan
           IF BisaMenyelesaikanPola(tipeTower, targetRuin) THEN
               PenyelesaianAkhirTower(tipeTower, targetRuin)
               targetRuin <- NULL // Misi Selesai
           ELSE
               // 2. Sketsa Cetak Biru Sekali
               IF BelumAdaTandaApapun() AND CatSisa >= 25 THEN
                   TandaiPola(tipeTower, targetRuin)
```

```
               // 3. Tembak Petak Kosong/Musuh Dekat
               IF AdaPetakUntukDiSerang() THEN
                   Serang(PetakTerdekat)
                   didAttack <- TRUE
               ELSE
                   didAttack <- FALSE

               // 4. Manuver Penghilang Jitter Buntu (Active Orbit)
               IF NOT didAttack AND BisaBergerak() THEN
                                                   ArahOrbitMenyudut   <-
Putar90Derajat(ArahKe(targetRuin))
                   IF BisaJalan(ArahOrbitMenyudut) THEN
                       Jalan(ArahOrbitMenyudut)
                   // Fallback jika tabrak teman/tembok
                   ELSE IF BisaJalan(LawanOrbitDuaKaliX) THEN
                       Jalan(LawanOrbitDuaKaliX)
       ELSE IF BisaBergerak() THEN
           moveWithAdjacencyPenalty(rc, ArahKe(targetRuin))

   // Prioritas 4: Menelusuri Area Lepas
   IF targetRuin == NULL AND BisaBergerak() THEN
       navigateGreedy(rc, FALSE) // Berburu wilayah
```

4.1.2.4 Splasher

Prosedur `runSplasher` direkayasa sebagai penopang alat pertahanan maupun senjata pamungkas penetrasi blok. Dikarenakan agen unit Area of Effect memercikkan air mahal secara masif, kami tak mendesain tembakan menyasar ubin netral paling banyak saja. Himpunan solusi serangan `runSplasher` memanipulasi skor prioritas ledakan secara rakus melalui Heuristik Bias Terpadu. Apabila radius ledakan mendempet bergesekan dengan sasaran titik target konstruksi Ruin milik sekutu, skor titik akan dilipatgandakan lebih dari +100 poin, menjadikan unit Splasher instan menembus kerumunan sebagai pahlawan yang menyiram sisa tanah kosong demi membangun Tower sekutu.

```
PROCEDURE runSplasher(rc):
   // Memutuskan Serangan Area Skor Maksimal
   // Pengaman +5 poin dari total biaya agar tidak macet di tengah
   IF BisaBeraksi() AND CatSekarang >= BiayaSerangan + 5 THEN
       skor_tertinggi <- -1.0
       target_terbaik <- NULL

          KANDIDAT_SERANG <-  Semua petak dalam matriks tembak (Radius
Tembak)
       FOR EACH target IN KANDIDAT_SERANG:
           skor <- 0.0
                                          FOR    EACH    ubin_radius    IN
LingkaranCipratan_Kedekatan_Radius4(target):
               IF ubin == KOSONG THEN skor <- skor + 1.0
```

```
                    ELSE IF ubin == MUSUH AND LetakTengahSasaran(target,
ubin) THEN
                        skor <- skor + 1.0 // Menimpa musuh di zona inti
ledakan

               // Heuristik Ekstrem: Dedisikasikan ledakan pada formasi
Menara
            IF targetRuin != NULL THEN
                jarakRadius <- JarakKuadrat(target, targetRuin)
                IF jarakRadius <= 4 THEN
                        skor <- skor + 100.0 // Sangat Greedy! Bersihkan
lahan!
                ELSE
                    skor <- skor + (25.0 / AkarKuadrat(jarakRadius))

            IF skor > skor_tertinggi THEN
                skor_tertinggi <- skor
                target_terbaik <- target

        IF target_terbaik != NULL AND skor_tertinggi >= 1.0 THEN
            SerangCipratanArea(target_terbaik)

   // Prosedur Navigasi Keberangkatan
   IF BahkanBisaBergerak() THEN
       navigateGreedy(rc, FALSE)
```

4.1.2.5 Helper Methods & Navigation

Sistem komputasi navigasi bot dipresentasikan dalam fungsi asisten yang menginjeksi vektor deterministik tanpa perulangan siklik sempit seperti A* Pathfinding. Dalam fungsi navigateGreedy, skor akhir setiap 8 arah mata angin digabungkan melalui nilai bobot: Inverse Distance Area mendorong robot menjelajahi titik kotor/kosong paling menguntungkan (nilai 1/akar(jarak)). Symmetri Pull menarik agen agar tak terlalu tersesat jauh di luar poros pusat cermin pertandingan. Hal utama dari struktur pergerakan kami bertengger pada sub-sistem Adjacency Penalty: Hukuman deduksi tajam sebesar -50 bobot jika bot memposisikan letak badannya bersebelahan dengan robot sekutu, mencerai-beraikan antrian macet di peta sempit.

```
FUNCTION navigateGreedy(rc, fleeMode):
   TitikPusatSimetri <- KoordinatSumbSiku(rc)
   skor_maks <- -999999.0
   arah_terbaik <- TENGAH

   FOR EACH arah IN ALL_DIRS:
       IF TidakBisaLewat(arah) THEN LanjutkanPencarian

       lokasi_depan <- LokasiSayaSekarang + arah
       skor <- 0.0

       // Bonus Konsisten Linear
```

```
        IF arah == ArahGiliranSebelumnya THEN skor <- skor + 10.0

        // Mode Pergi Jika Refill
        IF fleeMode == TRUE AND TerdeteksiBercakMusuh(lokasi_depan) THEN
            skor <- skor - 100.0

        // Adjacency Spacing Penalty: Hukum Jitter / Clumping Sekutu
        FOR EACH robot_sekutu IN RobotRadarKita:
            IF lokasi_depan BersebelahanDengan(robot_sekutu) THEN
                skor <- skor - 50.0 // Jauhi area dempetan ini

        // Komponen Tarikan Area Belum Tercat (Inversi Jarak)
        FOR EACH ubin IN UbinSensorMentah:
            IF ubin == KOSONG OR ubin == MUSUH THEN
                dist_sq <- JarakKuadrat(lokasi_depan, ubin.lokasi)
                IF dist_sq <= 20 AND dist_sq > 0 THEN
                    skor <- skor + (1.0 / AkarKuadrat(dist_sq))

        // Sentripetal Simetri Tarikan Basis
                      dist_sym   <-   Max(1,   JarakKuadrat(lokasi_depan,
TitikPusatSimetri))
        skor <- skor + (15.0 / AkarKuadrat(dist_sym))

        IF skor > skor_maks THEN
            skor_maks <- skor
            arah_terbaik <- arah

    IF arah_terbaik != TENGAH THEN
        BergerakKeArah(arah_terbaik)
        ArahGiliranSebelumnya <- arah_terbaik
```

4.1.3 Splasher specialist

4.1.3.1 Fungsi utama

Sistem dispatcher kami pada bot spesialis ini diimplementasikan dengan sangat ringkas melalui fungsi utama run. Bot mengidentifikasi tipe saat ia hidup dan mendelegasikan perintah pada prosedur `runSplasher` dan `runTower` menggunakan blok penanganan galat universal untuk berjaga dari `GameActionException` instan.

```
PROCEDURE run(RobotController rc):
   exploreDir <- PilihArahAcak(ALL_DIRS)
   CetakLog("Splasher Specialist Bot Starting...")

   WHILE True DO
       turnCount <- turnCount + 1
       TRY
           tipe <- rc.getType()
           IF tipe == SPLASHER THEN
```

```
                runSplasher(rc)
            ELSE IF tipe adalah TOWER THEN
                runTower(rc)
            ELSE
                // Fallback Control
                IF BisaBergerak() THEN
                    Bergerak(exploreDir)
        CATCH Exception
            CetakError()
        FINALLY
            Clock.yield()
```

4.1.3.2 Tower

Pada spesialisasi ini, runTower didedikasikan secara keras dan hanya mengeksekusi operasi acak terhadap rotasi mata angin untuk mencetak tentara murni.

```
PROCEDURE runTower(rc):
   arah_acak_pembangunan <- PilihArahAcak(ALL_DIRS)
            lokasi_target    <-    LokasiSekarang    +
arah_acak_pembangunan

    IF BisaBeraksi AND BisaMembangunRobot(SPLASHER,
lokasi_target) THEN
       BangunRobot(SPLASHER, lokasi_target)
```

4.1.3.3 Splasher

Logika putaran giliran bagi unit lapangan dikonfigurasi melalui prosedur runSplasher yang membungkus pemrosesan sensor sekali jalan, pelacakan Tower cat terdekat, prioritas perbekalan, aksi tembak terukur, dan akhirnya navigasi greedy.

```
PROCEDURE runSplasher(rc):
   // 1. Dapatkan informasi global dan filter state
   InfoUbin <- rc.senseNearbyMapInfos()
   InfoRobot <- rc.senseNearbyRobots()
   cat_sekarang <- rc.getPaint()
   batas_refill <- 20% KapasitasMaksimal

   updateNearestPaintTower(rc, InfoRobot)
```

```
     // Prioritas 0: Tarikan Cat Oportunis tanpa
relokasi
   IF cat_sekarang < 90% Kapasitas THEN
       tryRefill(rc, nearestPaintTower)
         cat_sekarang <- rc.getPaint() // Perbarui
angka cat terkini

   // Prioritas 1: Kebutuhan Refill Mandatori
    IF cat_sekarang < batas_refill OR cat_sekarang <
BiayaSerang (50) THEN
       IF nearestPaintTower != NULL THEN
           moveFuzzy(rc, ArahKe(nearestPaintTower))
       ELSE
                      greedyNavigation(rc,  InfoUbin,
LokasiSekarang) // Berburu tower/eksplor
       RETURN // Lewati tahap pengeboman

       //  Prioritas  2:  Greedy  Paint  Heuristic
(Attacking)
    IF BisaBeraksi AND cat_sekarang >= BiayaSerang
THEN
        KANDIDAT_UBIN <- array filter dari InfoUbin
(Kosong atau Musuh saja)
       target_terbaik <- NULL
       skor_maks <- -1

          TITIK_SARAN <- Semua koordinat di radius
serangan (DistSq <= 4)
       FOR EACH target IN TITIK_SARAN:
           skor <- 0
           FOR EACH ubin IN KANDIDAT_UBIN:
                     distRq <- JarakKuadrat(target,
ubin.Lokasi)
               IF distRq <= 4 THEN
                   IF ubin == KOSONG THEN
                       skor <- skor + 1
                    ELSE IF ubin == MUSUH AND distRq
<= 2 THEN
                           skor <- skor + 1 // Ubin
inti dapat merusak cat lawan

           IF skor > skor_maks THEN
               skor_maks <- skor
```

```
                target_terbaik <- target

            // Ambang Batas Pengontrol Pengeluaran
Berdasarkan Proporsi
         IF target_terbaik != NULL AND (skor_maks *
40 > rc.getPaint()) THEN
            Serang(target_terbaik)

   // Prioritas 3: Navigasi
   IF BisaBergerak THEN
                    greedyNavigation(rc,   InfoUbin,
LokasiSekarang)
```

4.1.3.4 Helper Methods & Navigation

Sistem komputasi asisten meliputi navigasi pencarian target berdasarkan proporsi invers kepadatan, serta helper fungsi penyerapan cat `(moveFuzzy)`.

```
FUNCTION       greedyNavigation(rc,       InfoUbin,
LokasiSekarang):
   // 1. Ambisi Penjelajahan Global ke Sarang Lawan
                           totalUbinKosong        <-
HitungSemuaUbinTarget(InfoUbin)
   IF totalUbinKosong < 5 THEN
           lokasi_simetri <- Titik(LebarPeta - 1 -
LokasiSekarang.X, Tinggi - 1 - LokasiSekarang.Y)
       moveFuzzy(rc, ArahKe(lokasi_simetri))
       RETURN

     // 2. Kalkulasi Kalkulus Jarak Penjelajahan
Lokal
   arah_terbaik <- TENGAH
   heuristik_maks <- -1.0

   FOR EACH arah IN ALL_DIRS:
       IF TidakBisaBergerak(arah) THEN CONTINUE

       lokasi_depan <- LokasiSekarang + arah
       heuristik_arah <- 0.0

       FOR EACH ubin IN InfoUbin:
           IF ubin == KOSONG OR ubin == MUSUH THEN
                distSq <- JarakKuadrat(lokasi_depan,
```

```
ubin.Lokasi)
               IF distSq > 0 AND distSq <= 20 THEN
                    // Kerapatan semakin tinggi jika
ubin target semakin dekat
                    heuristik_arah <- heuristik_arah
+ (1.0 / AkarKuadrat(distSq))

       IF heuristik_arah > heuristik_maks THEN
           heuristik_maks <- heuristik_arah
           arah_terbaik <- arah

     IF  arah_terbaik  !=  TENGAH  THEN  moveFuzzy(rc,
arah_terbaik)
   ELSE moveFuzzy(rc, ArahLacakAcakSebaran)

FUNCTION updateNearestPaintTower(rc, InfoRobot):
    // Filter semua "InfoRobot" kawan yang bertipe
"Tower".
     //  Modifikasi  nearestPaintTower  dengan  yang
nilai distSq terendah dari saya.

FUNCTION tryRefill(rc, towerLoc):
   IF towerLoc != NULL AND BisaBeraksi() THEN
        IF JarakKuadrat(LokasiSekarang, towerLoc) <=
2 THEN
              jumlah_tarikan <- KapasitasMaksimal -
CatSekarang
                      IF  jumlah_tarikan  >  0  AND
BisaTransfer(-jumlah_tarikan) THEN
                                TransferCat(towerLoc,
-jumlah_tarikan) // Tarik pakai argumen minus

FUNCTION moveFuzzy(rc, ArahTujuan):
     IF  TidakBisaBergerak  OR  ArahTujuan  ==  TENGAH
THEN RETURN
             IF    BisaBergerak(ArahTujuan)    THEN
Bergerak(ArahTujuan); RETURN

     IF  BisaBergerak(ArahTujuan.RotasiKanan1)  THEN
Bergerak(ArahTujuan.RotasiKanan1); RETURN
      IF  BisaBergerak(ArahTujuan.RotasiKiri1)  THEN
Bergerak(ArahTujuan.RotasiKiri1); RETURN
     IF  BisaBergerak(ArahTujuan.RotasiKanan2)  THEN
Bergerak(ArahTujuan.RotasiKanan2); RETURN
```

```
      IF  BisaBergerak(ArahTujuan.RotasiKiri2)  THEN
Bergerak(ArahTujuan.RotasiKiri2); RETURN
```

4.2 Penjelasan Struktur Data dan Fungsi

4.2.1 Main Bot

Secara umum, dalam kompetisi *Battlecode* 2025, *game engine* telah menyediakan sebagian besar struktur data internal melalui *Application Programming Interface* (API). Oleh karena itu, arsitektur mainbot sangat tergantung pada kelas bawaan dari pake `battlecode.common.*`. Meski demikian, ada beberapa penggunaan variabel *state* global sebagai struktur data caching dan status memori temporal.

4.2.1.1 Struktur Data

4.2.1.1.1 Struktur Data Bawaan Battlecode Engine

Nama Struktur	Tipe	Keterangan
`RobotInfo[]`	Array Objek Engine	Berisi informasi semua robot (kawan dan lawan) dalam jangkauan pandang sensor. Di-iterasi untuk memfilter target serangan, mendeteksi musuh, dan caching menara sekutu
`MapInfo[]`	Array Objek Engine	Memuat data setiap ubin yang terlihat: tipe cat (*PaintType*), keberadaan reruntuhan (*hasRuin*), dan tanda marker. Digunakan *Splasher* untuk kalkulasi *AoE* dan *Soldier* untuk pencarian *Ruin*.
`MapLocation[]`	Array Lokasi Engine	Dihasilkan oleh `rc.getAllLocationsWithinRadiusSquared()`. Dipakai *Splasher* untuk mengumpulkan semua kandidat titik tembak dalam radius 4.
`Direction[] ALL_DIRS`	Array Konstanta Statis	Berisi 8 arah mata angin: `NORTH`, `NORTHEAST`, `EAST`, `SOUTHEAST`, `SOUTH`, `SOUTHWEST`, `WEST`, `NORTHWEST`.

		Konstanta ini menjadi fondasi loop pencarian arah pada `moveFuzzy()` dan pencarian lokasi *spawn Tower*.

4.2.1.1.2 Variabel State Global

Nama Variabel	Tipe	Nilai Awal	Keterangan
Variabel Penghitung Giliran dan Navigasi			
`turnCount`	int	0	Penghitung giliran sejak robot lahir. Digunakan oleh `mopperPatrol` untuk menentukan kapan arah patroli perlu dirotasi (`patrolTick` % 6).
`patrolDir`	Direction	Arah Acak	Arah pergerakan patroli saat ini untuk unit *Mopper*. Dirotasi secara acak saat robot terjebak atau setelah 6 giliran.
`patrolTick`	int	0	Penghitung khusus giliran patroli *Mopper*.
`stuckCount`	int	0	Penghitung berapa kali robot gagal bergerak ke segala arah (terkunci penuh). Digunakan `mopperPatrol` untuk memicu pergantian arah patroli.
`isTracingRight`	boolean	true	Penanda arah rotasi pada algoritma `Bug0` Navigasi. true berarti robot menelusuri dinding ke kanan (searah jarum jam). Dibalik (`!isTracingRight`) jika robot terjebak > 15 giliran.
`traceTurns`	int	0	Penghitung berapa giliran robot sudah berada dalam mode menelusuri dinding. Direset setiap kali robot berhasil bergerak lurus ke arah tujuan.

Variabel Cache Lokasi Penting			
`homeBase`	MapLocation	-	Lokasi pertama kali robot di-spawn. Menjadi titik referensi mundur (*refill*) dan dasar kalkulasi simetri peta pada `estimateEnemyTower`. Hanya di-set sekali saat `run()` pertama kali dieksekusi.
`nearestAllyTower`	MapLocation	-	Cache lokasi menara sekutu yang paling dekat dari posisi robot saat ini. Diperbarui setiap giliran oleh `cacheAllyTower`. Digunakan *Mopper* sebagai titik batas *Leash*, Splasher untuk *refill* cat, dan semua unit untuk navigasi pulang.
`furthestAllyTower`	MapLocation	-	Cache lokasi menara sekutu yang paling jauh dari homeBase. Digunakan `mopperPatrol` sebagai basis perhitungan titik pos patroli di garis depan.
`targetEnemyTower`	MapLocation	-	Target global menara musuh. Di-set oleh `estimateEnemyTower` berdasarkan pengamatan langsung atau estimasi simetri peta, lalu di-broadcast kepada sesama unit sekutu. *Soldier* menggunakannya sebagai tujuan navigasi penyerangan.
Variabel State Konstruksi			
`currentRuin`	MapLocation	null	Reruntuhan yang sedang dikerjakan oleh unit *Soldier*. *Null* berarti sedang mencari target baru.
`ruinAttemptTurns`	int	0	Penghitung berapa giliran

			Soldier sudah tertahan di satu reruntuhan tanpa berhasil menyelesaikannya.
`ABANDON_THRESHOLD`	int	12	Batas waktu maksimal mengerjakan satu ruin. Jika `ruinAttemptTurns` >= 12, ruin tersebut ditinggalkan dan *Soldier* mencari target baru.
Variabel Statistik Spawn			
`moppersSpawned`	int	0	Total *Mopper* yang telah di-*spawn* oleh menara ini.
`splashersSpawned`	int	0	Total *Splasher* yang telah di-*spawn* oleh menara ini.
`soldiersSpawned`	int	0	Total *Soldier* yang telah di-*spawn* oleh menara ini.
Konstanta Ambang Batas			
`MOPPER_REFILL`	double	0.5	Batas cat (50% kapasitas) yang memicu *Mopper* untuk pulang dan isi ulang.
`SOLDIER_REFILL`	double	0.3	Batas cat (30% kapasitas) yang memicu *Soldier* untuk pulang dan isi ulang.
`SPLASHER_REFILL`	double	0.3	Batas cat (30% kapasitas) yang memicu *Splasher* untuk pulang dan isi ulang.
`LEASH_SQ`	int	130	Batas jarak kuadrat maksimal *Mopper* dari menara sekutu terdekat. Jika *Mopper* lebih jauh dari ini, ia dipaksa kembali (mekanisme *Leash*/Tali Pengikat).
Objek Pembantu Lain			
rng	Random	-	Objek *Random Number Generator* dengan *seed* tetap (42) untuk hasil deterministik. Digunakan untuk memilih arah patroli acak dan arah patrol

			awal.

4.2.1.2 Daftar Definisi dan Fungsi

Nama Fungsi	Keterangan
Fungsi Utama	
`run(RobotController rc)`	Titik masuk (*entry point*) utama. Berperan sebagai *dispatcher* yang memanggil fungsi spesifik unit berdasarkan `rc.getType()`.
`runSoldier(RobotController rc)`	Logika giliran menara: fase serangan (tembak musuh terdekat) lalu fase produksi (spawn unit sesuai rasio).
`runTower(RobotController rc)`	Logika giliran *Soldier* dengan hirarki prioritas: *refill* → selesaikan pola opportunis → bangun di *ruin* → serang musuh → eksplorasi.
`runSplasher(RobotController rc)`	Logika giliran *Splasher*: *refill* taktis → serangan AoE berbasis skor → navigasi menuju area kosong terpadat.
`runMopper(RobotController rc)`	Logika giliran *Mopper*: *refill* → cek *Leash* → serang target → hapus cat musuh → patroli.
Fungsi Seleksi dan Produksi	
`chooseSpawnType()`	Memilih tipe unit yang akan di-spawn berdasarkan rasio komposisi pasukan saat ini (target: 30% *Soldier*, 50% *Splasher*, 20% *Mopper*).
`findSafeRuinInSector(. . .)`	Mencari reruntuhan yang paling ideal untuk dibangun: aman dari *Splasher* musuh, berada di sektor peta yang benar, dan paling dekat.
`selectMopperTarget(. . .)`	Memilih target terbaik *Mopper*: prioritas utama *Splasher* musuh terdekat, cadangan musuh lain terdekat.
`nearestEnemy(. . .)`	Mencari robot musuh (bukan menara) yang paling dekat dari posisi Soldier.
`countEnemiesNear(. . .)`	Menghitung jumlah musuh dalam radius tertentu dari suatu titik, digunakan untuk memutuskan `MopSwing` vs. serangan tunggal.
Fungsi Konstruksi Tower	
`buildAtRuin(. . .)`	Eksekusi bertahap pembangunan tower di sebuah ruin: mendekat → tandai pola → selesaikan pola → cat ubin

	per-ubin.
`tryFinishAnyPattern(. . .)`	Secara oportunistik menyelesaikan pola tower mana saja yang hampir selesai dalam jangkauan pandang.
`tryFinishPattern(. . .)`	Versi terfokus dari `tryFinishAnyPattern`, hanya mengerjakan ruin yang sesuai dengan `targetRuin` tertentu.
Fungsi Navigasi	
moveFuzzy(RobotController rc, Direction dir)	Algoritma `Bug0` *Navigation*. Mencoba bergerak lurus; jika terhalang, menyusuri dinding dengan rotasi arah 45° hingga menemukan jalan.
`splasherNavigate(. . .)`	Navigasi khusus *Splasher*: bergerak menuju area berkonsentrasi ubin kosong/musuh menggunakan skor *inverse-distance*, atau rush ke titik simetri global jika peta kecil.
`mopperPatrol(. . .)`	Pergerakan patroli *Mopper* menuju titik pos penjagaan 40% antara menara terdepan dan pusat peta.
Fungsi Utilitas Pendukung	
`catchAllyTower(. . .)`	Memperbarui cache `nearestAllyTower` dan `furthestAllyTower` setiap giliran.
`estimateEnemyTower(. . .)`	Mendeteksi atau memperkirakan lokasi menara musuh berdasarkan sensor langsung atau simetri peta.
`doRefill(. . .) / tryRefill(. . .)`	Mengelola proses pulang dan pengisian ulang cat dari menara sekutu.
`tryPaintUnderfoot(. . .)`	Secara oportunistik mencat ubin di bawah kaki robot jika belum berwarna sekutu.
`engageMoppertarget(. . .)`	Mengeksekusi serangan *Mopper* terhadap target: `MopSwing` (AoE) jika musuh bergerombol, atau serangan tunggal jika tidak.
`nearestVisibleEnemyPaint(. . .)`	Mencari ubin bercat musuh yang paling dekat dalam pandangan sensor.
`broadcastMsg(. . .)`	Meng-*encode* koordinat lokasi sebagai integer dan mengirimkannya ke menara sekutu terdekat.
`decodeMsg(. . .)`	Mendekode pesan yang diterima menara dan memperbarui `targetEnemyTower`.

4.2.2 Snowball bot

Dalam mengimplementasikan solusi *greedy* berbias heuristik terdistribusi pada bot ini, kami menghindari struktur data yang rumit (seperti Graph, Tree dll) demi menekan penggunaan alokasi *bytecode* dan memori. Kami mengandalkan penyederhanaan struktur data primitif dan pemanfaatan paradigma pemrograman prosedural berbasis caching global. Berikut adalah rincian fungsionalitas dan entitas pembentuk arsitektur bot kami:

4.2.2.1 Struktur Data

4.2.2.1.1 Variabel Global Statis

Nama Variabel	Keterangan
`myLoc`	Menyimpan lokasi robot saat ini
`nearestPaintTower`	Menyimpan lokasi menara cat terdekat
`targetRuin`	Flag status apakah bot sedang dalam mode pengisian ulang cat darurat
`isRefilling`	Menyimpan memori arah pergerakan terakhir
`lastDirection`	Flag yang menahan produksi unit baru
`holdSpawnSignal`	Flag yang menahan produksi unit baru

Array 1 Dimensi		
Nama Struktur	Tipe Objek API	Keterangan
`nearbyTiles`	`MapInfo[]`	Array 1D yang menampung informasi petak peta (ubin) di sekitar robot.
`nearbyRobots`	`RobotInfo[]`	Array 1D yang menampung informasi robot sekutu/musuh di sekitar robot.
`nearbyRuins`	`MapLocation[]`	Array 1D yang menampung informasi lokasi reruntuhan di sekitar robot.

Bitmasking Skalar 32-Bit (Komunikasi)		
Bit Range	Fungsi / Data yang Disimpan	Contoh Penggunaan

4-bit pertama	Mendefinisikan Tipe Pesan (Message Type)	0x10000000 (Sinyal Musuh)
14-bit kedua	Menyimpan koordinat Sumbu-X	Lokasi target di sumbu horizontal
14-bit terakhir	Menyimpan koordinat Sumbu-Y	Lokasi target di sumbu vertikal

4.2.2.1.2 Daftar Definisi dan Fungsi

Nama Fungsi / Prosedur	Keterangan
Fungsi dan Prosedur Utama	
`run(RobotController rc)`	Prosedur sentral (*switchboard*) yang dipanggil tanpa henti dalam struktur blok `while(true)`. Menangani rekam jepret lingkungan utama *Global Sensing*), pembagian alur tugas (*branching*) berdasarkan identitas (tipe) unit, serta injeksi mekanisme interupsi tanggap darurat seperti transisi ke dalam mode pengisian ulang (*Refill Mode*).
`runTower(RobotController rc)`	Pilar prosedur spesialisasi fungsional Menara. Membungkus fungsi analitik greedy khusus menara.
`runSoldier(RobotController rc)`	Pilar prosedur spesialisasi fungsional *Soldier*. Menyimpan evaluasi fungsional fiksasi reruntuhan eksklusif dan algoritma *Active Orbiting*.
`runSplasher(RobotController rc)`	Pilar prosedur spesialisasi fungsional *Splasher*. Mengontrol navigasi menuju area kosong/musuh dominan.
Fungsi Helper Pengambilan Keputusan	
`decideSpawnType()`	Fungsi murni yang mengevaluasi heuristik rasio selisih *Splasher* dan *Soldier* guna memuntahkan jenis unit yang tepat di batas 250 atau 1000 chips. Mengembalikan enumerator `UnitType`.
`decideTowerToBuild()`	Mengeksekusi rasio formasi menara statis secara konstan dengan pola: *Money -> Paint -> Money -> Paint*.

Prosedur Logistik dan Komunikasi	
`handleRefill(RobotController rc)`	Berjalan secara otomatis sebelum agen mengevaluasi komando apa pun. Mengecek ambang batas toleransi cat dan secara sepihak menyita hak kendali gerak bot untuk pulang segera ke menara cat terdekat jika amunisi berada pada level krisis (*Hard Safety Buffer* 40 cat)
`updateNearestPaintTower(RobotController rc)`	Fungsi tracker untuk memperbarui memori radar menara secara greedy. Jika menara rumah hancur, robot mendaftarkan koordinat menara cat kawan baru dari *Sensory Cache*
`encodeMsg(. . .)`	Fungsi bedah kriptografi yang mengubah data koordinat dan tipe instruksi ke dalam *integer* 32-bit tunggal.
`decodeMsgType(. . .)`	Fungsi bedah kriptografi penerjemah tipe instruksi (4-bit pertama) dari *integer* bitmask.
`readMessages(RobotController rc)`	Menangkap permintaan relai intelijen komunikasi radio antar robot baris terdepan dan menara belakang.
Fungsi Eksekusi Navigasi	
`navigateGreedy(. . .)`	Setiap 8 arah dievaluasi berdasarkan momentum, pemindaian rintangan, *Symmetry Pull*, inversi jarak target bidik, hingga mode pelarian (*fleeing*).
`moveWithAdjacencyPenalty(. . .)`	Fungsi *mikro-manuver* terarah pengganti jalur lurus. Alihkan jalur diskon tajam bobot nilai langkah apa pun yang akan menempatkan unit bersebelahan paralel dengan sekutu, menjamin kelancaran "F*luid Navigation*" serta mengeliminir sumbatan lalu lintas (*traffic block*).
`isObstacle(. . .)`	Predikat logika *boolean* pembantu untuk memvalidasi kelayakan titik ubin, menghindari halangan ilegal.
`hasTower(. . .)`	Predikat logika *boolean* pembantu memastikan pergerakan maupun tembakan tidak menabrak tower yang ada.

4.2.3 Splasher specialist

Dalam mengimplementasikan spesialisasi Soldier pada bot ini, kami mengandalkan pemanfaatan memori fungsional berbasis caching state global yang sangat ringan guna mempertahankan orientasi konstruksi dan momentum jelajah tanpa perlu menggunakan manajemen pathfinding (graf) yang memakan memori. Berikut adalah rincian fungsionalitas dan entitas pembentuk arsitektur bot spesialis Soldier:

Nama Variabel	Keterangan
`targetRuin`	Memori penahan (lock-on cache) untuk mengamankan satu target reruntuhan eksklusif yang sedang dihampiri atau dikerjakan oleh unit. Bernilai null jika bot sedang menganggur mencari reruntuhan baru.
`nearestPaintTower`	Menyimpan koordinat menara cat sekutu terdekat hasil pindahan radar sensor. Titik ini menjadi titik evakuasi pengisian ulang (*Refill Point)*.
`lastDirection`	Menyimpan rekaman memori dari arah pergerakan terakhir yang berhasil dilewati robot. Digunakan sebagai dasar kalkulasi Massive Bonus Momentum guna membasmi fenomena gerak jiggle (maju-mundur).
`turnCount`	Penghitung siklus total berjalannya waktu giliran sejak instansi robot dimunculkan ke peta.
`rng`	Objek Random Number Generator bawaan utilitas Java sebagai basis nilai entropi gerakan paksa saat robot terkunci di gang sempit.

Array 1 Dimensi		
Nama Struktur	Tipe Objek API	Keterangan
`nearbyTiles`	`MapInfo[]`	Array 1D yang menampung informasi petak peta (terlewat dapat dilewati/tidak, bekas reruntuhan, status pewarnaan) untuk evaluasi medan gerak.
`nearbyRobots`	`RobotInfo[]`	Array 1D relaksasi loop yang menampung informasi unit kawan dan musuh di radius

		pandang. Digunakan untuk meraba posisi menara suplai dan menara musuh di atas ruins.
`nearbyRuins`	`MapLocation[]`	Array 1D yang menyajikan pangkalan data mentah kordinat semua reruntuhan dalam jangkauan radar visibilitas (distance squared 20).

4.2.3.1.1 Daftar Definisi dan Fungsi

Nama Fungsi / Prosedur	Keterangan
Fungsi dan Spesialisasi Cabang	
`run(RobotController rc)`	Prosedur switchboard pusat (pusat kendali infinite loop). Menyelimuti tangkapan exception (Anti-Crash) dan mendelegasikan eksekusi giliran secara primitif berdasar entitas genetik: Soldier atau Tower.
`runTower(RobotController rc)`	Pilar prosedur singular spesialisasi fasilitas perbanyakan yang berjalan serakah (greedy). Menara tidak mempertimbangkan formasi bertahan, tugasnya hanyalah meramu mutlak bibit pasuka Soldier secepat-cepatnya ke arah mata angin acak.
`runSoldier(RobotController rc)`	Jantung komputasi dari unit mekanik utama Soldier. Terdiri dari susunan tumpukan prioritas ketat (Strict Rule Based): Opportunistic Refill, Greedy Breadcrumbing (mengecat lantai selagi melintas murni oportunis), Infrastructure Heuristic Builder, Pengahancuran Menara Musuh yang menduduki Ruin, dan Scouting pemetaan simetris.
Fungsi Helper Keputusan Konstruksi	
`hasTower(...)`	Predikat asisten bertipe boolean untuk mengecek tumpukan array array radar bot dan memvalidasi apakah secara sah Ruin yang dituju masih "telanjang" atau telah diklaim pondasi infrastrukturnya oleh fasilitas robot lain.

`determineTowerType(..)`	Prosedur pengambilan keputusan wujud ekonomi infrastruktur (Greedy Tower Type Selection). Berevolusi berdasarkan proporsi jumlah menara (modulus parity): Menara perdana dan kedua didedikasikan mutlak untuk Money Tower, lalu diseimbangkan silih berganti (selang-seling pingpong) antara uang dan cat.
Prosedur Logistik dan Komunikasi	
`handleRefill(RobotController rc)`	Berjalan secara otomatis sebelum agen mengevaluasi komando apa pun. Mengecek ambang batas toleransi cat dan secara sepihak menyita hak kendali gerak bot untuk pulang segera ke menara cat terdekat jika amunisi berada pada level krisis (*Hard Safety Buffer* 40 cat)
`updateNearestPaintTower(..)`	Fungsi pembaru memory-tracker untuk membaca indeks array nearbyRobots kawan yang bertipe instalasi pertahanan dan memateri kalkulasi Euclidean (jarak kuadratik radius terdekat) murni.
`tryRefill(..)`	Logika penyedotan sisa cairan. Robot mengkalkulasi celah perut muatannya dan memberikan lemparan transfer perintah dengan nominal negatif (-drawAmount) terhadap Tower penyedia agar cat instan tersedot langsung ke tubuh unit tersebut.
Fungsi Eksekusi Navigasi	
`greedyNavigation(. . .)`	Motor utama penggerak jelajah scouting. Mengalkulasikan kompilasi tiga bobot sekaligus terhadap 8 bukaan mata angin: Inverse dari densitas lahan telanjang kosong (Math.sqrt), Massive Momentum Bias 2.0x pada arah lastDirection, dan tarikan manipulasi global ke titik balik cermin (Symmetry Pull) untuk menerkam pangkal peneluran lawan (`symmetryLoc`).
`moveFuzzy(. . .)`	Algoritma obstacle avoidance (Bug0 Pattern). Jika unit terancam mendabrak dinding di arahan vertikal lurus kompas pandangannya, kalkulasi merotasikan kompas sebesasr 45% serong ke

	kanan/kiri (`rotateRight / rotateLeft`), dan memori lolos gerak instan diukir pada state `lastDirection`.

4.3 Pengujian Greedy

Untuk menguji ketangguhan dan efektivitas algoritma *greedy* yang telah kami rancang, kami melakukan serangkaian simulasi pertandingan antar bot (Main Bot, Alternatif 1, dan Alternatif 2). Kami menetapkan metodologi pengujian terkontrol dengan memilih tiga jenis peta (*maps*) yang merepresentasikan karakteristik medan tempur yang berbeda, yaitu:

1. Small Map (20x20) : Mewakili kondisi pertempuran jarak dekat yang sangat cepat (*fast-paced*), di mana efisiensi *spawn* awal sangat menentukan.
2. Medium Map (40x40) : Mewakili kondisi medan yang penuh dengan tembok dan reruntuhan untuk menguji kecerdasan navigasi *Bug0* dalam menghindari hambatan.
3. Large Map (60x60) : Mewakili kondisi ekonomi jangka panjang, di mana heuristik rasio populasi dan strategi pembangunan menara sangat diuji.

Pertandingan dilakukan dalam format 1 vs 1 hingga batas maksimal 2000 ronde. Berikut adalah ringkasan hasil pengujian kami :

Nama Map	Pemenang	Alasan
mainbot vs snowball bot		
DefaultSmall	Snowball (round 304)	mainbot ngecat 70% lebih dari mapnya
CastleDefense	Snowball (round 212)	mainbot ngecat 70% lebih dari mapnya
DefaultHuge	Snowball (tiebreaker)	snowball ngecat lebih banyak dengan coverage 63.8%
mainbot vs splasher specialist		
DefaultSmall	splasher_specialist (round 232)	splasher_specialist ngecat 70% dari mapnya
CastleDefense	mainbot (round 1672)	mainbot ngecat 70% dari mapnya
DefaultHuge	mainbot (tiebreaker)	mainbot ngecat lebih banyak dengan coverage 38.2%
snowball vs splasher specialist		
DefaultSmall	snowball (round 1067)	snowball ngecat lebih banyak dengan coverage 67.7%

CastleDefense	snowball (round 223)	snowball ngecat lebih banyak dengan coverage 71.8%
DefaultHuge	snowball (round 1865)	snowball ngecat 70% dari mapnya

4.4 Analisis Hasil Pengujian

4.4.1 Analisis mainbot

Mainbot, yang merepresentasikan heuristik lokal/absolut (*strict priority*) bertumpu kuat pada fiksasi reruntuhan dan pertarungan musuh terdekat. Evaluasi kaku yang dipaksakan mainbot dari *Splasher Specialist* di lokasi-lokasi peta luas (*CastleDefense* dan DefaultHuge), Mainbot menderita ketertinggalan efisiensi waktu terhadap taktik seimbang dan cair milik *Snowball*. Evaluasi kaku yang dipaksakan *mainbot* (contoh pertahanan kaku pada poin abandon *threshold*) rupanya menyebabkan kerugian komputasi giliran ketika target terus bergeser secara dinamis yang diperparah pada kondisi peta-peta *tiebreaker* melebihi 1000 giliran.

4.4.2 Analisis snowball

Strategi *Balanced Strategic Swarm* milik *Snowball Bot* terbukti menjadi pendekatan *greedy* yang paling matang, terbukti dari kekonsistenannya memenangkan *seed* di seluruh ukuran peta. Keberhasilan strategi ini berakar pada pengaturan logistik yang dinamis (spawn throttle) dan fungsi *mikro-manuver* `moveWithAdjacencyPenalty`. Fungsi ini mencegah terjadinya fenomena penyumbatan jalan (*traffic bottleneck*) antar sekutu, sehingga perluasan cat ke seluruh peta terjadi sangat optimal dibandingkan varian bot lain yang robotnya sering menabrak kawan sendiri.

4.4.3 Analisis splasher_specialist

Strategi *Splasher Specialist* sangat mematikan di peta berukuran kecil (*DefaultSmall*), terbukti dapat mengalahkan *mainbot* dominan di ronde 232 karena daya hancur area (*Area-of-Effect*) unit *Splasher* mampu mewarnai persentase sempit peta kecil secara drastis dalam beberapa giliran saja. Akan tetapi, heuristik ini kedodoran pada peta-peta raksasa seperti *DefaultHuge*. *Splasher* adalah unit boros cat dan lambat sehingga pada ruang lintasan yang jauh menuju base musuh, mereka sering kehabisan cat sebelum tiba dan mudah di-counter melalui laju penyebaran cepat varian standar.

BAB V : Kesimpulan dan Saran

5.1 Kesimpulan

Dari proses perancangan, pembuatan kode, sampai pengujian bot kami (Mainbot, Snowball, dan Splasher Specialist) di Battlecode 2025, ada beberapa kesimpulan yang bisa kami tarik:

1. Algoritma Greedy Sangat cocok untuk Batasan Ketat
   Di Battlecode, memori dan bytecode sangat dibatasi (hanya 17.500 instructions per ronde). Daripada menggunakan algoritma pencarian jalur yang rumit dan berat, pendekatan Greedy terbukti sangat efektif. Dengan memecah keputusan menjadi evaluasi per giliran (micro-decisions), bot kami bisa menjalankan puluhan fungsi tanpa takut terkena timeout.
2. Pentingnya mengatur ruang gerak tim
   Awalnya, pendekatan Greedy biasa membuat bot kami sering bertabrakan satu sama lain karena masing-masing hanya memikirkan targetnya sendiri (egois). Namun, setelah kami menambahkan aturan Adjacency Penalty (pengurangan nilai jika bergerak terlalu dekat dengan teman) pada Snowball bot, pergerakan tim menjadi jauh lebih rapi. Pasukan menyebar dengan sendirinya seperti swarm yang cerdas sehingga area peta bisa diwarnai lebih cepat.
3. Tidak ada strategi sempurna untuk semua map
   Walaupun strategi Snowball (yang seimbang) paling sering menang, performa bot ternyata sangat bergantung pada ukuran peta. Contohnya, Splasher Specialist sangat mematikan di peta kecil karena bisa menutupi area dengan cepat. Tapi di peta besar, strategi ini gagal berantakan karena boros cat dan lambat jalannya. Ini membuktikan kalau nilai heuristik Greedy yang kaku tidak bisa diandalkan untuk semua jenis pertempuran.

5.2 Saran

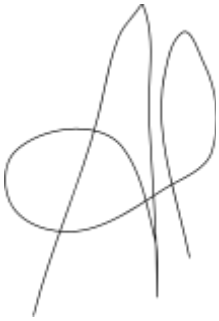
Dalam pengembangan bot Battlecode ini, kami menyadari masih banyak hal yang bisa diperbaiki untuk turnamen atau tugas selanjutnya:

1. Menggunakan Heuristik yang Dinamis
   Saat ini, variabel pengambil keputusan di bot kami (seperti rasio *spawn* unit) masih statis atau di-*hardcode*. Ke depannya, akan lebih baik kalau bot bisa beradaptasi sendiri dengan kondisi game. Misalnya, bot otomatis memprioritaskan pembuatan unit pekerja kalau penguasaan peta di bawah 20% pada ronde-ronde awal.
2. Menambahkan Pathfinding yang Ringan
   Navigasi bot kami saat ini murni menggunakan gaya Fuzzy/Bug0 yang kadang bisa terjebak kalau map-nya terlalu berliku seperti labirin. Mungkin bisa ditambahkan algoritma pathfinding seperti A, tapi yang sudah disederhanakan (*micro-pathfinding*). Algoritma ini cukup dijalankan sesekali saja saat bot benar-benar mentok, supaya bytecode tetap aman.
3. Memaksimalkan Komunikasi Radio (*Bitmasking*)
   Sistem komunikasi 32-bit integer yang kami buat sudah berhasil mengatur perintah dasar. Untuk pengembangan selanjutnya, sisa ruang bit yang ada bisa dipakai untuk hal yang lebih kompleks, misalnya mengirimkan sinyal peringatan bahaya (*Area-of-Effect*) dari

musuh. Dengan begitu, pasukan di depan bisa otomatis menghindar sebelum serangan musuh jatuh.

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Al Farabi

Fahd Muhammad Zahid

LAMPIRAN

Pranala GitHub: https://github.com/fahdmz/Tubes1_idk-gnh

DAFTAR PUSTAKA

1. GeeksforGeeks. (2025). *A* Search Algorithm*. Diakses pada 13 Maret 2026, dari https://www.geeksforgeeks.org/dsa/a-search-algorithm/
2. Battlecode. (2025). *Battlecode 2025 Java API Documentation – battlecode.common package*. Diakses pada 13 Maret 2026, dari https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html
3. XSquare. (2025). *A Guide to Battlecode*. Diakses pada 13 Maret 2026, dari https://battlecode.org/assets/files/battlecode-guide-xsquare.pdf
4. Rinaldi Munir. (2026). *Algoritma Greedy (Bagian 1)*. Bahan Kuliah IF2211 Strategi Algoritma. Institut Teknologi Bandung. Diakses pada 13 Maret 2026 dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf
5. Rinaldi Munir. (2026). *Algoritma Greedy (Bagian 2)*. Bahan Kuliah IF2211 Strategi Algoritma. Institut Teknologi Bandung. Diakses pada 13 Maret 2026 dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/05-Algoritma-Greedy-(2026)-Bag2.pdf
6. Rinaldi Munir. (2026). *Algoritma Greedy (Bagian 3)*. Bahan Kuliah IF2211 Strategi Algoritma. Institut Teknologi Bandung. Diakses pada 13 Maret 2026 dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/06-Algoritma-Greedy-(2026)-Bag3.pdf